Efficiently Linking Unstructured Data for Multi-step Reasoning

Jiaming Liang
liangjm@seas.upenn.edu
University of Pennsylvania
Philadelphia, PA, USA

Haydn Jones
haydnj@seas.upenn.edu
University of Pennsylvania
Philadelphia, PA, USA

Jacob R. Gardner
jacobrg@cis.upenn.edu
University of Pennsylvania
Philadelphia, PA, USA

Mark Yatskar
myatskar@cis.upenn.edu
University of Pennsylvania
Philadelphia, PA, USA

Zachary Ives
zives@cis.upenn.edu
University of Pennsylvania
Philadelphia, PA, USA

Abstract

Modern LLMs and AI agents are transforming data engineering by enabling multi-step reasoning over unstructured content. This shift has prompted the incorporation of LLM capabilities into SQL DBMSs via *semantic operators* [23, 29], which extend the relational algebra by replacing Boolean predicates with LLM prompts. Such operators facilitate complex queries that identify semantically related objects across disparate sources, enabling novel agentic discovery tools that synthesize pharmaceutical candidates from evidence in the published literature and experimental datasets.

These workloads require the joint execution of multi-attribute filtering, multi-vector search, and both exact and approximate semantic joins. In this work, we develop a unified approach to *multi-step reasoning queries* over structured attributes, multiple vectors, and semantic and exact joins, using a combination of novel algorithmic and indexing techniques exploiting the property that close matches are a tail event in the similarity score distribution. Our method, DASE, jointly optimizes attribute filtering and vector similarity scoring across tables. We validate our work across existing benchmarks and novel scientific workloads.

PVLDB Reference Format:
Jiaming Liang, Haydn Jones, Jacob R. Gardner, Mark Yatskar, and Zachary Ives. Efficiently Linking Unstructured Data for Multi-step Reasoning. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://anonymous.4open.science/r/DASE-2C61.

1 Introduction

Novel AI and database systems have emerged that construct answers by integrating information from multiple unstructured sources, using LLM- and agent-based reasoning over their semantic content and relationships. Examples include "deep research" [12, 50], "wide search" tools [4, 13, 17, 47, 48], "co-scientists," LLM-based information extraction tools [1, 38], and unstructured-to-structured query systems based on *semantic operators* [23, 31, 36, 40]. Despite their architectural differences, these systems share a common methodology: given a user query, they decompose it into plans, perform ranked LLM-driven retrieval and extraction over heterogeneous sources, and assemble answers via ranked linking and joining of intermediate results. We view this emerging class of systems as instances of *agentic data engineering*. Most such systems incorporate *local content indexing and retrieval*, for "retrieval augmented generation" (RAG) over relevant textual passages or knowledge graphs [10, 33, 37]. Additional data may come from persistent "memory" [6, 22]; both results and memory are combined in the prompt using "context engineering" [25].

Querying over RAG, knowledge graphs, and "memory" resembles the data lake integration problem [26], extended to unstructured documents. An emerging use case lies in AI for science, where we must *integrate* disparate facts available in the literature, assembling tabular results. In recent work [15], we found that scientific data available in structured datasets is dwarfed by what can be extracted from the literature; moreover, the literature can capture nuances such as how data was collected in different studies. Consider a real-world example drawn from our application:

Example 1.1 (Agentic data engineering for molecular discovery.). Our team, in collaboration with chemists and life scientists, is building an agentic data engineering "deep research" tool for discovering candidate molecules with pharmaceutical benefits. The process of discovering molecules inherently requires a **comprehensive hybrid search** model to navigate the complexity of scientific data. The data comprises both raw text and extracted knowledge graph-like annotations from papers in PubMed (https://pubmed.ncbi.nlm.nih.gov/). Queries typically return sets of papers or descriptors, which satisfy a combination of approximate- and exact-match predicates along criteria such as molecular sequence, target organism, description, and toxicity. A molecule's structure can be encoded as a string [46] or bit vector [35], or mapped to a specialized embedding [7]. Accompanying textual descriptions might also be embedded using a text embedding. In answering a question about a molecule's properties, evidence is often fragmented even within a single document across paragraphs. Thus, a query about a molecule might (1) approximately match against a ChemBERTA embedding, extracted from the text into a knowledge graph, but linked back to the paragraph in which it appears; (2) match exactly on an extracted `organism`; (3) match against embeddings of experimental conclusions. All of these matches must be joined to ensure they are within

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

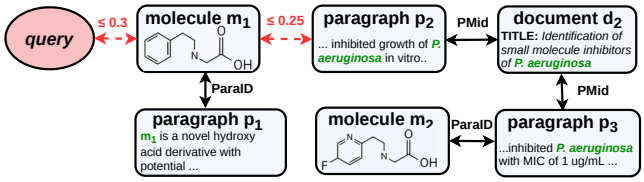

Figure 1: Multi-step reasoning query in Example 1.2: solid lines represent exact joins; dashed lines represent similarity joins. Results are ranked via distances on the dashed lines.

a common paper; and ranked using a function that **combines all of the individual matches**.

Example 1.2. Building upon Example 1.1, a scientific-discovery query might be based on the observation that small molecules may have similar characteristics even if they are not particularly similar in ChemBERTA embedding space. Thus we might seek to *find related molecules*, by matching on a first document, then **linking and joining via text description embedding** with *other* documents, then traversing (joining) to their extracted molecular descriptions. Figure 1 visualizes this.

Given that the above workload requires joins to link together different components, such as annotations, paragraphs, and papers, is not well-suited to conventional vector DBMSs such as Pinecone or Weaviate [34, 45]: such systems only do approximate nearest-neighbor (ANN)-based lookups of content along a *single* embedding, requiring an external query processor to do nested-loops join-style lookups to stitch together results. Even the Milvus [44] database, designed for hybrid search, focuses on logical predicates with approximate match, but does not support joins. Thus, although we include Milvus as a baseline in our experiments in this paper, we have concluded that the best overall alternative is a relational DBMS with approximate-nearest-neighbor (ANN) vector index support, which has advantages in terms of flexibility, standard query language support, and more. As we demonstrate in Section 5, standard RDBMSs, even with vector indexing, perform poorly for our workload as well as other semantic reasoning tasks [18]: when given a hybrid query with logical filter predicates and approximate match, neither "filter-first" techniques using a B+ tree to evaluate the predicates, nor using approximate-nearest-neighbor indices first to retrieve approximate matches [11, 24], works well. Information-retrieval-style reranking [3, 27] often suffers from low recall. Multi-step reasoning queries introduce new challenges in query indexing, optimization, and execution.This paper makes the following contributions:

- We formalize **multi-step reasoning queries**, which integrate multi-attribute filtering, multi-vector similarity scoring, and cross-table joins into a unified query model.
- We develop a novel indexing method, SemJI, that allows efficient exploration of candidates in top-k multi-step reasoning queries.
- We extend the ANN index support in PostgreSQL to support effective integration with SemJI.
- We propose a ***D**ata **A**gent **S**imilarity-search **E**ngine*, which jointly optimizes attribute predicates and vector similarity scoring across relational joins, providing a starting point for future research.
- We evaluate over existing semantic benchmarks, as well as a novel benchmark that captures multi-step reasoning queries reflective of scientific discovery.

To ensure reproducibility, our code, data, and full technical report are available at https://anonymous.4open.science/r/DASE-2C61/.

2 Multi-Step Reasoning Queries

This paper targets query answering tasks that involve filtering and integrating content extracted from unstructured sources, into a local DBMS with both approximate nearest-neighbor search over embeddings, as well as logical predicates over relationships.

Data model. Our underlying data model comprises a set of object-relational tables, each of which includes scalar attributes, embeddings, and BLOBs representing raw document and file content. Tables are linked through foreign key relationships. Using a combination of user-defined functions and semantic operators [29, 38], BLOB data may be parsed using standard document parsing tools, mapped to an embedding, and then analyzed using LLM operators such as *semantic map* to extract structured tuples from the parsed content; *semantic filter* to test tuples' content against a prompt, etc.

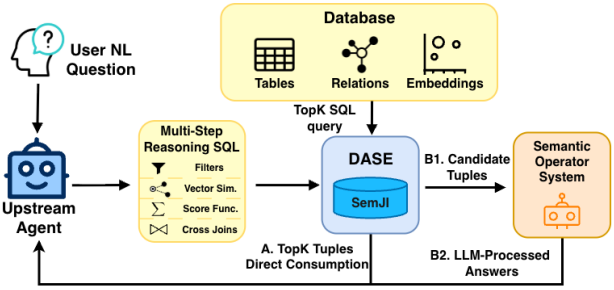

Figure 2: DASE interactions with system components

2.1 Top-k Multi-Step Reasoning Queries

Our target use case involves an upper-layer agentic system, performing deep or wide search, or another form of multi-step reasoning over a user's natural-language question. This decomposes the user request into a structured query over a local dataset. The query combines multi-attribute filtering, multi-vector similarity scoring, and cross-table joins, together with a scoring function that aggregates these signals into a unified ranking objective. DASE executes this query and returns the top-k results. We assume the scoring function is supplied by the upstream agent. DASE is agnostic to its origin; in our own research agent, the weights are fit on a *calibration set* [29] of labeled samples via logistic regression, which satisfies Fagin's monotonicity assumption [8] and thus admits efficient threshold-based top-k evaluation.

The top-k stream produced by DASE is consumed by the upstream agent in one of two modes:

1. Direct consumption. When the agent's question reduces to finding the best-matching combinations of evidence—a typical pattern in wide search and multi-objective discovery—the ranked output is the answer. Cross-encoders [27] are a common way for the agent to map per-source signals into a unified ranking.

2. Prefilter for semantic operators. When the agent's question requires reasoning that goes beyond ranking raw rows—e.g., judging whether a paragraph actually supports a claim, deciding if two entities refer to the same concept, or synthesizing an answer from extracted evidence—the top-k output is not the final answer but a

high-recall candidate set passed to a downstream semantic operator system [29, 36, 40] for LLM-based evaluation. Best practice [29] is to split each semantic operation into a cheap "prefilter" stage that prunes via proxy signals such as embedding distance, followed by the actual LLM call only on survivors. The prefilter's threshold is set over a calibration set [29]. In multi-step reasoning, this prefilter is exactly a ranked, thresholded select-join-union query over embedding distances and structured attributes—the workload DASE is built for.

In both modes, DASE sees the same problem: execute a top-k query whose scoring function is given by the upstream agent, over data that mixes structured attributes, multiple embedding spaces, and cross-table joins.

We now formalize the top-k multi-step reasoning query.

2.1.1 Query Model. A multi-step reasoning query Q is formally defined by the triple $(\mathcal{P}, \mathcal{S}, \mathcal{J})$, which characterizes the feasibility space, ranking objectives, and relational dependencies.

Predicates ($\mathcal{P}$): $\mathcal{P} = \{p_1, \ldots, p_{|\mathcal{P}|}\}$ is a set of heterogeneous predicates over 1 or more **structured components** (which may be the results of a join or Cartesian product). This set may be empty ($\mathcal{P} = \emptyset$), indicating an unconstrained search space. Regardless of their specific semantic types (such as range constraints, set memberships, or probabilistic filters) each $p_i \in \mathcal{P}$ can be conceptually modeled as a Boolean indicator function $p_i : \mathcal{R} \to \{0, 1\}$. For a given query subexpression q and any resulting row instance $r \in q(\cdot)$, $p_i(r)$ logically evaluates to 1 if r satisfies the constraint, and 0 otherwise.

Similarity Join Conditions ($\mathcal{J}$): $\mathcal{J} = \{(e_{1,i}, e_{2,i}, dis_i, \tau_i)\}_{i=1}^{|\mathcal{J}|}$ is an optional set of relational constraints ($\mathcal{J} = \emptyset$ denotes a standard single-table query). Each condition specifies a semantic relatedness join between **unstructured components** across tables, where a link is established if the distance between their respective embeddings satisfies $dis_i(e_{1,i}, e_{2,i}) \leq \tau_i$.

Scoring Model ($\mathcal{S}$): $\mathcal{S} = \text{score}_f(s_1, \ldots, s_{|\mathcal{S}|})$ defines the unified ranking objective, over a **multi-step reasoning query**. To guarantee a definable Top-k retrieval, the query must specify at least one scoring signal ($|\mathcal{S}| \geq 1$); the aggregation function f can in its simplest form be an identity mapping over a single signal (e.g., $f(s_1) = 1 \cdot s_1$). For more complex queries, each scoring component s_i represents a quantitative signal belonging to one of three categories: (1) *semantic similarity*: the distance $dis(\mathbf{q}_v, e_i)$ between a query vector $\mathbf{q}_v$ and a target embedding e_i; (2) *relational semantic linkage distance*: the proximity $dis(e_{1,i}, e_{2,j})$ between instance embeddings from potentially disparate tables; (3) *structural attribute* distance: a ranking signal derived directly from a scalar attribute a_i. The aggregation function f synthesizes these signals into a unified metric. The goal is to retrieve the top-k tuples that satisfy $\mathcal{P}$ and $\mathcal{J}$ while maximizing $\mathcal{S}$.

As discussed above, the scoring signals and weights are provided by an upper-layer agentic system (e.g., through calibration, learned rankings [5, 21, 49], etc.) or by direct user specification; our proposed methods are fundamentally agnostic to their origin.

Our task fundamentally extends classic rank aggregation [8, 41]. Traditional settings find top answers by aggregating independent scores that evaluate the query against individual records. In contrast, our core challenge is computing dynamic, interdependent scores to evaluate semantic join conditions on the fly. Reconciling these highly coupled data-to-data affinities with structured logical predicates—while avoiding an exhaustive search space—is the primary computational hurdle in retrieving the top-k results.

```
SELECT m1.smiles_string AS source_molecule,
    m2.smiles_string AS related_molecule,
    d2.pubmed_id AS related_paper,
     (0.6 * (m1.chemberta_embed <=> $molecule_embed) +
     0.4 * (p2.text_embed <=> m1.description_embed)) AS
     semantic_similarity
FROM molecule m1
    JOIN paragraph p1 ON m1.paragraph_id = p1.
     paragraph_id
    JOIN paragraph p2 ON p2.text_embed <=> m1.
     description_embed < 0.25
    JOIN document d2 ON p2.pubmed_id = d2.pubmed_id
    JOIN paragraph p3 ON d2.pubmed_id = p3.pubmed_id
    JOIN molecule m2 ON p3.paragraph_id = m2.paragraph_id
WHERE  m1.chemberta_embed <=> $molecule_embed < 0.3
    AND p1.document_id <> d2.pubmed_id
    AND m1.molecule_id <> m2.molecule_id
    ORDER BY semantic_similarity DESC LIMIT 100;
```

Listing 1: Molecular relationship query for `$molecule_embed`

Example 2.1. Listing 1 shows the SQL for Example 1.2. Results are ordered by `semantic_similarity`, which is a weighted sum with calibrated weights on the terms across multiple distance components including both lookup distance and join distance.

Standard ANN indices fall short here for two reasons. First, as they are implemented in engines such as PostgreSQL, they are designed for 1-to-N point lookups, making them **unable to execute M-to-N semantic joins following a block nested loops pattern**. Instead, they essentially only support tuple nested loops joins. Second, because they **cannot jointly evaluate the multi-signal scoring function**, the true top results for the overall query might be buried arbitrarily deep within the ANN's ranked stream, precluding early termination. We thus develop an alternative approach, starting with the notion of **precomputing and caching** partial answers based on distributional knowledge, and expanding our answer frontier as needed. This decouples join discovery from scoring and supporting threshold algorithms [8] for earlier termination.

2.2 Caching Semantic Join Tail-Events

Traditional relational joins consider a Cartesian product of input pairs, and filter those matching a predicate. Much of the algorithmic innovation lies in effective pruning and in minimizing I/Os or random accesses. In contrast, evaluating semantic join conditions requires computing continuous distances between vector pairs, which, without a means of filtering, intuitively threatens to degenerate into a dense Cartesian product of computations and results. However, analyzing the geometric properties of high-dimensional embeddings reveals that semantic retrieval is fundamentally a sparse, "tail-event" problem. That, in turn, leads to an opportunity to precompute and materialize the tail events.

High-dimensional distance distribution. Most embeddings (e.g., from OpenAI [28] or Google [19]) are normalized to the unit hypersphere, satisfying $\|u\|_2 = 1$. For two unit vectors $u, v \in \mathbb{S}^{d-1}$, the squared Euclidean distance $\|u - v\|_2^2$ must lie within $[0, 2]$.

According to the concentration of measure phenomenon on the sphere [43]: in high-dimensional spaces like embeddings, the inner product $u \cdot v$ of two randomly chosen vectors tightly concentrates around 0, following a normal distribution $\mathcal{N}(0, 1/d)$. By the Central Limit Theorem (CLT), as $d \to \infty$, the distribution of pairwise distances converges to a Gaussian. For squared distances between unit vectors, the probability mass heavily concentrates around the mean $\mu = 2$, and variance vanishes as dimensionality increases.

Semantic joins as rare tail-events. The geometric implication here is that the vast majority of arbitrary vector pairs exhibit an "average" distance near μ, representing zero semantic affinity. The semantically meaningful *near-neighbors* occupy only the extreme leftmost tail of this Gaussian curve:

$$P(d_2 < \epsilon) = \int_0^{\epsilon} f(d_2)\, \mathrm{d}d_2$$

Because this integral yields an exponentially small probability for small ϵ, the number of highly relevant semantic pairs is much smaller than the Cartesian product. While this theoretical model assumes dimensional independence, empirical real-world data closely aligns with it. As shown in Figure 3, the pairwise distance distribution of IMDB actor and director embeddings forms a Bell curve where the bulk of distances fall between 0.55 and 0.85, leaving only a tiny fraction as valid semantic joins.

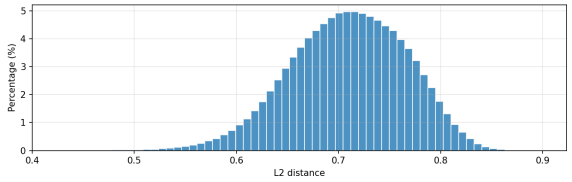

Figure 3: L2 distance distribution: pairwise among IMDB Actor_Director embeddings (Gemini-embedding-001)

Implications. In certain settings, e.g., those in which predicates are applied to restrict the query to semantically related items, embedding distances may not fully satisfy the Gaussian distribution described above, due to *manifold clustering* [39]. Nonetheless, as we experimentally validate, optimizing for the common case, and having a fallback strategy for other cases, is beneficial. We make two architectural decisions.

- **Semantic join index as a cache for tail events:** In top-k semantic search, we seek to differentiate *unusually similar* pairs from the dense group of "average" distances, i.e., the statistical tail constitutes the goal. These rare, highly affiliated pairs represent the most viable candidates for the top-k results, short-circuiting evaluation of the full Cartesian. In fact, analogous to a traditional join index [42], we can efficiently pre-compute and cache only this extreme tail to safely bound the search space.
- **Need for batched retrieval interfaces:** Modern ANN index implementations are tailored for single-vector lookups and lack native mechanisms to ingest a stream of candidate nodes (such as those provided by our proposed join index) for batched evaluation. Our implementation must specifically address this gap to evaluate interdependent scoring components efficiently.

Finally, to accommodate cases in which insufficient data is in the index, or where manifold clustering makes indexing cost-prohibitive, Section 4 presents fallback and adaptation strategies.

3 Semantic Join Index

Building on the strategy of caching semantic tail-events, we propose the **Semantic Join Index (SemJI)**. SemJI materializes closely affiliated vector pairs, directly instantiating the semantic join topology $\mathcal{J}$ of our query model. SemJI bypasses nearest-neighbor index lookups, providing greater performance and cost-based reasoning.

3.1 Motivation and Definition

In multi-step reasoning queries, synthesizing a logical chain inherently requires evaluating both single-hop and multi-hop similarity predicates. Because computing even a single semantic join against a nearest-neighbor index is costly, executing deep traversals becomes intractable. The SemJI **physically materializes the $\mathcal{J}$ operator** to break this bottleneck. It is formally defined as a set of 5-tuples:

$$(id_A, tid_A, id_B, tid_B, \mathrm{dist}(A, B))$$

where $\mathrm{dist}(A, B) \leq \tau$. The **distance threshold** τ defines the logical boundary of the materialized connectivity; any pair exceeding τ is considered outside the current discovery horizon. Each component maps the physical storage directly to the logical Q model:

- **Field Identifiers (id_A, id_B):** Specify the source embedding columns to map logical join components.
- **Tuple Pointers (tid_A, tid_B):** Serve as physical addresses for rapid attribute retrieval during predicate ($\mathcal{P}$) evaluation.
- **Interaction Signal (dist(A, B)):** Stores the pre-computed distance to provide the scoring component $\mathcal{S}$ with immediate input, decoupling join discovery from vector re-computation.

Beyond SemJI's ability to cache direct links, it also enables **transitive** evidence synthesis. To resolve multi-hop similarity predicates dictated by an agent's reasoning path, the query engine simply chains these 5-tuples using their physical pointers (e.g., joining tid_B of one hop to tid_A of the next), weighting and combining their thresholds to determine the overall search radius. This allows complex multi-hop discoveries to be executed purely through standard, highly optimized relational join operators. We describe query evaluation in Section 4.

3.2 Index Construction

The construction of the SemJI balances the requirement for high-recall connectivity against the computational constraints of massive datasets. We provide two construction pathways: an exact method ensuring zero false negatives for high-integrity workloads, and an approximate, index-agnostic method leveraging standard ANN structures for scalability.

Option 1: Exact construction. To guarantee the absolute integrity of the similarity join operator (i.e., zero false negatives), we can employ an **exhaustive Cartesian materialization** strategy. This process iterates over the Cartesian product of the embedding spaces to identify pairs satisfying the distance threshold τ. Formally, for two sets of embeddings $\mathcal{E}_A$ and $\mathcal{E}_B$, the construction materializes:

$$\mathcal{J}_{exact} = \{(e_i, e_j) \in \mathcal{E}_A \times \mathcal{E}_B \mid \mathrm{dist}(e_i, e_j) \leq \tau\} \quad (1)$$

This method serves as the ground-truth baseline, ensuring that the materialized connectivity graph $\mathcal{J}$ captures every valid semantic link within the defined discovery horizon.

Option 2: Approximate Construction for Scaling. For large-scale scenarios, we employ an asymmetric probe strategy to approximate the semantic join $\mathcal{E}_A \bowtie_\tau \mathcal{E}_B$. Our architecture incorporates a pluggable ANN interface. The system iterates over the smaller table (probe side) and queries an underlying vector index built on the larger table (build side). Any ANN implementation supporting radius or range filtering can be integrated as the candidate generator. To demonstrate this flexibility, we implement two variants:

E2LSH via filter-and-refine. For Euclidean (L_2) distance, we utilize Locality Sensitive Hashing (E2LSH) based on p-stable distributions. Rather than treating LSH as a definitive filter, we deploy it as a high-recall candidate generator in a standard filter-and-refine pipeline. We calibrate the quantization width to $w \approx 4\tau$ to aggressively minimize false negatives during the hash probe. The query engine then computes the exact distance for all collision candidates, eliminating false positives to strictly enforce the dist $\leq \tau$ condition.

Radius-expanded HNSW (Algorithm 1). Graph-based indices like HNSW provide state-of-the-art recall, but their standard implementations are intrinsically optimized for top-k retrieval. This can cause premature pruning of valid candidates that lie within τ but are locally strictly worse than the current beam. To adapt HNSW for SemJI construction, we implement a **radius-expanded traversal**. The traversal maintains a dynamic candidate buffer $\mathcal{W}$ of size ef but relaxes the pruning threshold (`gate`) to be the maximum of the ef-th distance and the index radius capacity τ_{cap}:

$$\text{gate} = \max(\mathcal{W}.\text{furthest}(), \tau_{cap}) \tag{2}$$

This structural invariant forces the traversal to explore all nodes within the target geometric region τ_{cap} regardless of local neighbor density, securing high recall for range queries. The adapted procedure is detailed in Algorithm 1.

Algorithm 1 Layer Traversal in Radius-Expanded HNSW

Require: Query q, Entry Point ep, Beam ef, Radius τ_{thre}, Cap τ_{cap}
Ensure: Result Set $\mathcal{R}$

$C \leftarrow \text{MinHeap}(ep)$; $\mathcal{W} \leftarrow \text{MaxHeap}(ep)$
$\mathcal{R} \leftarrow \emptyset$
while C is not empty **do**
  $c \leftarrow C.\text{pop}()$; $w \leftarrow \mathcal{W}.\text{peek}()$
  if $c.dist > w.dist$ **and** $c.dist > \tau_{cap}$ **then**
    break ▷ Prune only if outside both beam and radius
  end if
  $gate \leftarrow \max(\mathcal{W}.\text{peek}().dist, \tau_{cap})$
  for $e \in \text{neighbors}(c)$ **do**
    $d_e \leftarrow \text{dist}(e, q)$
    if $d_e \leq gate$ **or** $|\mathcal{W}| < ef$ **then**
      $C.\text{push}(e, d_e)$; $\mathcal{W}.\text{push}(e, d_e)$
      if $|\mathcal{W}| > ef$ **then** $\mathcal{W}.\text{pop\_furthest}()$
      end if
      if $d_e \leq \tau_{thre}$ **then** $\mathcal{R}.\text{add}(e)$
      end if
    end if
  end for
end while
return $\mathcal{R}$

3.3 Batched Execution Optimization

To mitigate the latency of random memory access and maximize arithmetic intensity in creating a SemJI, both construction pathways leverage **batched execution**. Instead of processing query vectors individually, the system processes a batch of queries $\mathcal{B}_q = \{q_1, \ldots, q_B\}$ simultaneously.

- **Exact Construction (Block-Nested Loop):** We implement the Cartesian product using a Block-Nested Loop (BNL) strategy. The embedding matrices are partitioned into fixed-size blocks suitable for L2/L3 cache residency. By loading a batch of probe vectors and comparing them against a block of build vectors, we utilize SIMD-accelerated linear algebra kernels to compute $B \times B$ distances in parallel. A vectorized filter then applies the threshold τ, minimizing memory writes by only materializing surviving pairs.
- **Approximate Construction (Shared Traversal):** In Radius-HNSW, spatially proximal queries in a batch tend to share traversal paths in the upper layers of the hierarchical graph. By synchronizing their descent, the system significantly improves CPU cache hit rates for the graph adjacency lists. Similarly, for E2LSH, queries are sorted by hash bucket ID, allowing the system to verify all candidates in a bucket with a single sequential read, reducing the stall cycles associated with pointer chasing.

3.4 Multi-Step Reasoning

Beyond caching pairwise similarity, SemJI naturally extends to support **multi-hop similarity**, effectively enabling complex multi-step fuzzy join paths (e.g., $A \xrightarrow{\tau_1} B \xrightarrow{\tau_2} C$) without runtime exhaustive search. By treating the materialized connectivity $\mathcal{J}$ as a relational view, the system can navigate deep query topologies simply by composing basic index lookups.

Composition Logic. An agent's multi-step reasoning dependency is satisfied by executing an equijoin sequence over the respective SemJI partitions. Formally, given a chain of semantic join conditions $\mathcal{J}_{chain} = \{(\mathcal{J}_1, \tau_1), \ldots, (\mathcal{J}_m, \tau_m)\}$, the set of valid tuple chains is derived symbolically as:

$$\mathcal{R}_{chain} \leftarrow \text{Fetch}(\mathcal{J}_1, \tau_1) \bowtie \text{Fetch}(\mathcal{J}_2, \tau_2) \bowtie \ldots \bowtie \text{Fetch}(\mathcal{J}_m, \tau_m) \tag{3}$$

where $\text{Fetch}(\mathcal{J}_i, \tau_i)$ returns the subset of the index satisfying the runtime distance threshold τ_i (where $\tau_i \leq \tau_{cap}$).

Predicate generation via semijoin reduction. Crucially, this composition allows the system to flatten complex topological constraints into a simple existence predicate, functionally serving as a **semantic semijoin reduction**. The result $\mathcal{R}_{chain}$ identifies exactly which base entities participate in a valid multi-hop path. We define the **join indicator predicate** $P_{\mathcal{J}}$ for a target table T as:

$$P_{\mathcal{J}}(t) = \mathbb{I}\left[t.id \in \Pi_T(\mathcal{R}_{chain})\right] \tag{4}$$

This derivation serves as a fundamental building block for the execution layer (detailed in Section 4.2 and Section 4.3). It allows the query engine to dynamically transform complex cross-table

dependencies into efficient, single-table Boolean filters ($\mathcal{P}$) before computationally expensive scoring operations.

3.5 Threshold Determination and Maintenance

Constructing a high-value Semantic Join Index (SemJI) requires addressing two primary challenges: (1) defining the boundary of the materialized cache (i.e., the distance threshold τ), and (2) handling, through an exception case, queries that require exploration beyond this pre-computed horizon. For index tuning, we employ a statistically grounded and workload-aware management strategy.

3.5.1 Statistical initialization. Building on the geometric properties established in Section 2.2, semantically related item pairs are expected to reside in the leftmost tail of the distance distribution. To instantiate τ, the system undergoes a warmup stage upon table registration. The system samples a representative subset of data to estimate the empirical mean μ and standard deviation σ of the pairwise embeddings. To ensure the index remains highly selective and memory-efficient, the default threshold is defined at the statistical tail, typically $\tau = \mu - 3\sigma$. This bounds the index size while guaranteeing the capture of semantic linkages. If a query workload is available: for each query with a threshold τ' on the join distance, we will raise τ until it meets or exceeds τ'.

3.5.2 Exhaustion and fallback. A critical invariant of SemJI is that any valid path step not present in the materialized index must lie within (or beyond) the dense mid-section of the distance Bell curve. When a SemJI stream exhausts its pre-computed entries, the system performs a graceful fallback to a reranking baseline. This baseline uses the existing threshold τ as a strict upper bound to prune the search space, ensuring that accuracy is maintained even when the query frontier exceeds the cache capacity.

3.5.3 Data evolution. As the database absorbs new entities and query workloads shift, we manage the evolution of SemJI through two complementary mechanisms.

Instance-level incremental updates: For insertions and deletions, the system performs localized maintenance. Any new embedding is probed against the existing index using the current threshold τ. This isolated approach ensures that individual writes remain lightweight and do not trigger expensive global recalculations.

Global-level workload evolution: To handle macro-level data drift, the system monitors the fallback rate (the frequency at which queries exhaust the cache) and triggers reranking. If this rate exceeds a predefined tolerance, indicating a structural shift in the distance manifold, the system triggers an asynchronous background re-estimation of μ and σ. When an updated threshold requires expansion ($\tau_{old} \rightarrow \tau_{new}$), the system avoids full re-computation by exploiting Algorithm 1. The currently materialized pairs already cache the terminal state of the prior graph traversal. Expansion is achieved by initializing the HNSW traversal queues with these existing boundary points and exploring outward to τ_{new}.

4 DASE Execution Strategy

The execution layer of DASE consists of three core components that interoperate to tackle the problems of multi-step reasoning with both similarity and Boolean predicates. The **Multi-Scoring Aggregation Engine** acts as the root operator, driving multiple

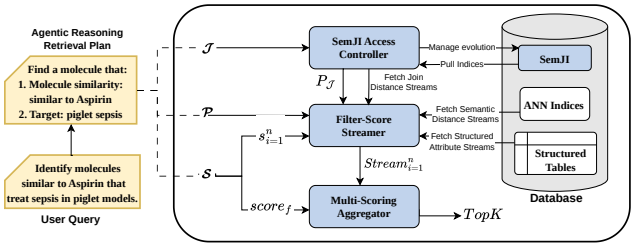

Figure 4: DASE architecture

Filtered-Score Streamer while the **Relational Index Manager** maintains the underlying connectivity state. Additionally, we have implemented a predicate-aware ANN index traversal strategy that considers predicates and distance at the same time. This works as an alternative to predicate-first and vector-first strategies, achieving optimal performance under certain conditions. Our approach requires a **monotonic scoring function** [8].

4.1 Predicate-aware ANN Index Traversal

Most hybrid search systems split their strategies into two camps: *vector-first* (post-filtering) and *filter-first* (pre-filtering). This creates a performance gap: vector-first works well when most items match the filter, while filter-first is better when the filter eliminates almost everything. Between these two extremes lies a "middle ground" where neither approach is efficient. To handle this, we implemented a **Predicate-aware ANN Index Traversal** that checks filters while walking the index graph. **Often one of the filter conditions is whether the ANN node endpoints lie within a batch of path endpoints retrieved from the SemJI index.**

Simulating an Oracle Graph. Our method is inspired by the *Oracle Graph* idea from ACORN [32]. Conceptually, if we had a custom index built only from the items that pass the filter, search would be optimal. Since we cannot build a new index for every query, we simulate this experience using the global HNSW index.

Transitive neighbor expansion. The main problem with walking a global ANN graph under constraints is that a path to a good candidate might be “blocked” by a node that fails the filter. To fix this, we modify the greedy search: When the immediate neighbors of a node are all filtered out, we look at their neighbors (2 hops away). By using these invalid nodes as bridges rather than dead ends, we can jump over gaps in the graph and find the true nearest neighbors that satisfy the query.

Bloom filter optimization. As noted above, DASE will frequently combine SemJI results with an ANN traversal — doing both a similarity join over a similarity-based lookup. Checking the filter condition for every node during a graph traversal is too slow if it requires fetching data from the main table. To speed this up, we compile the query’s node filter set into a **Bloom filter** [2] before the search begins. This allows the traversal engine to check if a node is valid using a fast bitwise operation. While the Bloom filter is fast, it consumes $O(\frac{N}{HashBeamSzie})$ memory and takes time to build. If the dataset is too large, the system falls back to standard disk-based strategies to avoid crashing.

4.2 Filtered-Score Streamer

The **Filtered-Score Streamer (FSS)** is the generates a sorted stream for **one** scoring signal under constraints. Formally, the input to FSS

is (1) a set of predicates $\mathcal{P}$ (2) a scoring signal s, which is one of: (a) semantic similarity, (b) semantic linkage distance, or (c) structured attribute, according our *Query Model* in Section. 2. The output of FSS will be an iterable streamer that streams entries which pass the filters in descending order of the scoring signal s. Symbolically, $FSS(\mathcal{P}, s) \rightarrow Stream((id_i, score_i))$. The FSS adapts its execution path based on the data source and predicate selectivity σ.

(a) Semantic Path: For the case of the scoring signal being semantic query distance $\text{dist}(\mathbf{q}, e_i)$, the scorer selects between attribute-first filtering, vector-first ANN traversal and above mentioned predicate-aware ANN traversal based on σ to minimize distance computation overhead. To be specific, when the selectivity σ is less than a threshold σ_1, we prioritize attribute-first, as the highly-selective filters prune out most of the candidates and give a short list of possible candidates that allow following one by one distance comparison to be efficient; For $\sigma > \sigma_2$, we prioritize vector-first strategy as most entities can pass the loss constraints, with a moderate expansion of K and post-filtering, we can retrieve the true top K. When $\sigma_1 < \sigma < \sigma_2$, we opt for the above mentioned predicate-aware index traversal.

Selection of σ thresholds. We resolve the σ threshold selection by offline profiling and cost-model calibration. For each workload, we generate multiple synthetic K-nearest neighbor search queries with filters of random selectivity, then apply the three search options: (1) vector-first, (2) attribute-first, and (3) predicate-aware ANN, tuning their parameters to achieve a fixed target recall (e.g., 0.98). We then generate a performance plot of selectivity vs. throughput and combine the curves of the three strategies. Due to the specialized efficiency of each strategy in its respective domain—attribute-first dominating in high-selectivity regimes (low σ), vector-first in low-selectivity regimes (high σ), and predicate-aware traversal in the intermediate zone—the combined throughput envelope exhibits a distinct "three-peak" pattern. Consequently, we choose the performance crossover points of these curves to be the thresholds σ_1 and σ_2 for runtime application.

(b) Relational Path: For linkage distance $\text{dist}(A, B)$, the scorer scans the **join index** in sorted order of the precomputed link distances, while applying $\mathcal{P}$ to the filtering attributes cached in the join index or entity metadata. This can be accelerated by traditional B-tree indices on the join index.

(c) Attribute Path: For a structured attribute scoring signal, we perform filtering with B-tree indices over the relational data.

4.3 Multi-Scoring Aggregator

The **Multi-Scoring Aggregator (MSA)** synthesizes disparate score streams under a unified ranking, based on an aggregation function $score_f$. To ensure the correctness of early termination, we require $score_f$ to be monotonic. The MSA adapts the Threshold Algorithm (TA) [8] and the Rank-Join Algorithm [20] to handle both multi-attribute scoring and joins. As we describe below, we extend TA with an adaptive strategy for advancing streams.

Definition. Formally, the MSA accepts n independent sorted streams $\{\mathcal{S}_i\}_{i=1}^n$ produced by upstream Filtered-Score Streamers (FSS), a monotonic aggregation function $score_f$, and a retrieval budget K. The operator outputs the top-K tuples that maximize $score_f$.

Algorithm 2 Multi-Scoring Aggregator (MSA) Execution

Require: Sorted Streams $\{\mathcal{S}_1, \ldots, \mathcal{S}_n\}$, Function $score_f$, Budget K
Ensure: Top-K result tuples $\mathcal{R}$

```
Q ← PriorityQueue()
L ← [∞] × n
τ ← ∞
while |Q| < K or Q.min_score() < τ do
    for i ← 1 to n do
        (e, s) ← S_i.next()
        L[i] ← s
        T_e ← FetchValidTuples(e)
        for t ∈ T_e do
            if t not seen then
                t.score ← score_f(t.s_1, ..., t.s_n)
                Q.push(t, t.score)
                Mark t as seen
            end if
        end for
    end for
    τ ← score_f(L[1], ..., L[n])
end while
return Q.pop_top(K)
```

Symbolically: $\text{MSA}(\{\mathcal{S}_i\}_{i=1}^n, score_f, K) \rightarrow \{c_1, \ldots, c_K\}$, where $\{c_i\}$ represents the set of valid tuples maximizing the objective function.

Implementation. The MSA executes by incrementally extending the search frontier across all input streams in a synchronized manner. In every iteration, the operator advances the cursors of the input streams (e.g., in a round-robin fashion) to fetch the next highest-scoring entry e from each stream. The detailed procedure is outlined in Algorithm 2.For each newly retrieved entry e, the MSA immediately triggers Random Access to materialize all valid tuples $\mathcal{T}_e$ that contain e. This step involves querying the Join Index or fetching sibling attributes to reconstruct complete candidate tuples. Any valid tuple involving e is discovered and evaluated the moment e is first encountered in *any* stream.

Any valid tuple that has *not* yet been materialized must consist entirely of components that lie **below** the current scan depth of all input streams. Consequently, the score of any unseen tuple is mathematically strictly bounded by the aggregation of the current cursor scores. The system maintains a global threshold τ, calculated as the aggregation of the last seen scores from every stream: $\tau = score_f(s_1^{\text{last}}, \ldots, s_n^{\text{last}})$. The algorithm terminates immediately when the K-th best candidate in the buffer satisfies $score(c_K) \geq \tau$, as no unseen combination of components can exceed this threshold.

Distribution-Aware Stream Selection. Recall from Section 4.3 that the MSA halts when the worst score among our current top-K candidates (denoted as θ_K) catches up to the global upper bound of unseen scores (τ). To stop as early as possible, we simply need to close this termination gap, $G = \tau - \theta_K$, down to zero. The round-robin approach in the standard Threshold Algorithm advances all streams equally. In practice, real-world data is heavily heterogeneous. Because streams have different score variances, density peaks, and aggregation weights, the actual scan depth required from each stream to reach the stopping condition varies wildly. To

minimize I/O overhead, we want to actively drive the algorithm towards its termination criteria. Therefore, at each iteration, instead of advancing blindly, we dynamically select and extend the specific stream i^* that offers the largest expected reduction in the termination gap, denoted as $\mathbb{E}[\Delta G_i]$. To analytically compute this expectation, we must model the distribution of the stream scores. As discussed in Sec. 2.2, in vector similarity search, distance scores will, in most cases, approximately follow a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$. Based on this premise, we can mathematically formulate the expected marginal benefit of advancing any given stream.

Mathematical Formulation. Without loss of generality, we assume a linear aggregation function $score_f = \sum_{i=1}^{n} w_i s_i$. For any stream i, let f_i and F_i denote the Probability Density Function (PDF) and Cumulative Distribution Function (CDF) of its corresponding Gaussian distribution. Let $L[i]$ represent its current cursor value, and N be the total cardinality of the database. Advancing a stream reduces the termination gap from two directions: depressing the global upper bound τ (via sequential access progression), and elevating the K-th score θ_K (via random access candidate materialization).

Theorem 4.1 (Expected Threshold Reduction). *The expected reduction in the global threshold τ achieved by advancing stream i is asymptotically inversely proportional to its local probability density:*

$$\mathbb{E}[\Delta\tau_i] \approx \frac{w_i}{N \cdot f_i(L[i])} \tag{5}$$

Proof. The threshold τ is strictly a function of the cursors $L[1], \ldots, L[n]$. Advancing stream i updates $L[i]_{old}$ to $L[i]_{new}$ in descending order. Based on the asymptotic theory of order statistics, the expected distance between adjacent order statistics at value x for a sample of size N drawn from distribution f is $\mathbb{E}[x_k - x_{k+1}] \approx \frac{1}{Nf(x)}$. Therefore, the expected drop in the weighted cursor is $w_i \mathbb{E}[L[i]_{old} - L[i]_{new}] = \frac{w_i}{Nf_i(L[i])}$. □

Theorem 4.2 (Expected Candidate Elevation). *Assume a bi-stream setting ($n = 2$) with streams i and j. Upon fetching a new entry from stream i with score s_i, the expected elevation of the K-th score θ_K via Random Access materialization is:*

$$\mathbb{E}[\Delta\theta_{K,i}] = \int_{s_{target}}^{L[j]} \left(w_i s_i + w_j x - \theta_K\right) \frac{f_j(x)}{F_j(L[j])} dx \tag{6}$$

where $s_{target} = \frac{\theta_K - w_i s_i}{w_j}$.

Proof. A newly fetched entry from stream i triggers a Random Access to stream j to retrieve its unobserved component x_j. Because x_j has not been encountered during the sequential scan of stream j, it is strictly bounded by the current cursor: $x_j \le L[j]$. Thus, the unobserved x_j follows a right-truncated Gaussian distribution with PDF $\frac{f_j(x)}{F_j(L[j])}$.

The new fully materialized score is $S = w_i s_i + w_j x_j$. The K-th score is elevated if and only if $S > \theta_K$, which requires $x_j > s_{target}$. The expected positive difference $\mathbb{E}[\max(0, S - \theta_K)]$ evaluated over the truncated support yields the integral. □

Theorem 4.3 (Optimal Greedy Stream Selection). *The optimal policy that maximizes the single-step expected reduction of the termination gap G selects the stream i^* such that:*

$$i^* = \arg\max_{i \in \{1,\ldots,n\}} \left(\mathbb{E}[\Delta\tau_i] + \mathbb{E}[\Delta\theta_{K,i}]\right) \tag{7}$$

Algorithm 3 Distribution-Aware Stream Selector

Require: Cursors L, Gaussian Densities f and F, Current θ_K, Weights w, Cardinality N
Ensure: Stream index i^* to advance

```
max_benefit ← −1
i* ← 1
for i ← 1 to n do
    E_τ ← w_i / (N · f_i(L[i]))                          ▷ Theorem 4.1
    E_θ ← 0
    s_i ← PeekNextScore(S_i)
    for j ≠ i do   ▷ Pairwise candidate elevation per Theorem 4.2
        s_target ← (θ_K − w_i s_i) / w_j
        if s_target < L[j] then
            E_θ ← E_θ + ∫_{s_target}^{L[j]} (w_i s_i + w_j x − θ_K) f_j(x)/F_j(L[j]) dx
        end if
    end for
    benefit ← E_τ + E_θ                                  ▷ Theorem 4.3
    if benefit > max_benefit then
        max_benefit ← benefit
        i* ← i
    end if
end for
return i*
```

Proof. By the linearity of expectation, the expected reduction of the gap is $\mathbb{E}[\Delta G_i] = \mathbb{E}[\Delta\tau_i - (-\Delta\theta_{K,i})] = \mathbb{E}[\Delta\tau_i] + \mathbb{E}[\Delta\theta_{K,i}]$. Selecting i^* trivially maximizes this local marginal benefit, reducing the threshold and elevating a candidate into a singular dimensional metric (score expectation) without relying on heuristic weights. □

Implementation. Theorems 4.1 through 4.3 guide the algorithm to aggressively pursue the optimal stopping coordinate. To operationalize this Distribution-Aware Stream Selection, the `for` loop in the Multi-Scoring Aggregator is replaced with a dynamic expectation evaluator. By utilizing $O(1)$ precomputed CDF and PDF lookup tables for the Gaussian distribution, the integration in Theorem 4.2 takes constant-time. Algorithm 3 outlines the pseudo-code for this stream selection subroutine, which is invoked at the start of each iteration to determine the most beneficial stream to advance.

4.4 SemJI Access Controller

The **SemJI Access Controller** functions as the storage abstraction layer that exposes the materialized semantic connectivity of SemJI to the query pipeline. It serves two distinct query-time roles depending on whether the relational data is treated as a hard constraint or a ranking signal. Additionally, it acts as the runtime monitor that drives the structural evolution of the index.

Materializing join predicates. When the query includes hard relational constraints ($\mathcal{J}$), the controller's objective is to define the valid search space. It retrieves the set of materialized pairs for each join condition and, in the case of multi-hop dependencies, executes an intersection of these sets to synthesize a unified **candidate set** of valid tuples. This process effectively converts topological constraints into a flat filter predicate $P_{\mathcal{J}}$, which is pushed down to

Table 1: Characterization of query workload classes.

WL	$\mathcal{P}$	Multi-$\mathcal{S}$	$\mathcal{J}$	Pattern	Expression
Intra-Entity Retrieval					
W_1	×	×	×	Semantic retrieval	$TopK\{FSS(\emptyset, s)\}$
W_2	✓	×	×	Filtered retrieval	$TopK\{FSS(\mathcal{P}, s)\}$
W_3	×	✓	×	Multi-scoring retrieval	$MSA(\{FSS(\emptyset, s_i)\}_{i=1}^{n}, score_f, K)$
W_4	✓	✓	×	Filtered multi-scoring	$MSA(\{FSS(\mathcal{P}, s_i)\}_{i=1}^{n}, score_f, K)$
Inter-Entity Synthesis					
W_5	×	×	✓	Semantic join	$TopK\{FSS(P_{\mathcal{J}}, s)\}$
W_6	✓	×	✓	Filtered join	$TopK\{FSS(\mathcal{P} \cup P_{\mathcal{J}}, s)\}$
W_7	×	✓	✓	Multi-scoring join	$MSA(\{FSS(P_{\mathcal{J}}, s_i)\}_{i=1}^{n}, score_f, K)$
W_8	✓	✓	✓	Filtered multi-scoring join	$MSA(\{FSS(\mathcal{P} \cup P_{\mathcal{J}}, s_i)\}_{i=1}^{n}, score_f, K)$

the *Filter-Score Streamer* to prune the search space. Symbolically, $P_{\mathcal{J}} \leftarrow\!\bowtie_{i=1}^{|\mathcal{J}|} \text{SemJIFetch}(\mathcal{J}_i)$.

Streaming relational scores. When the query utilizes relational proximity as a ranking objective (i.e., a scoring component $s \in \mathcal{S}$ is defined by the semantic link distance), the controller acts as a streaming source. It scans the underlying index to yield pairs (id, dis) sorted by their precomputed semantic distance. This stream is fed directly into the *Multi-Scoring Aggregator* as a standard ranking signal: $\mathcal{S}_{rel} \leftarrow \text{SemJIFetch}(\mathcal{J}_S)$.

Workload monitoring and evolution trigger. As described in Sec. 3.5.3, the controller performs workload monitoring. During query execution, it tracks the *fallback rate* (the frequency at which the materialized SemJI stream is exhausted). If the query engine consistently hits this cache boundary and triggers the reranking baseline, the controller asynchronously signals the maintenance module. This triggers the global-level boundary expansion ($\tau_{old} \rightarrow \tau_{new}$) using the Radius-Expanded HNSW traversal, ensuring the index evolves without bottlenecking real-time execution.

5 Experiments

Datasets. To evaluate DASE on deep research reasoning tasks, we constructed two benchmark datasets. In **IMDB**, base tables from the official IMDB (https://datasets.imdbws.com) are augmented with structured fields from the repository `jquigl/imdb-genres`. To support cross-table movie comparisons, the data is split by release year. In **MOLECULE**, we collected query templates from our agentic AI system built on a PubMed-derived molecule database (https://pubmed.ncbi.nlm.nih.gov/). These templates are used to simulate multi-step reasoning workflows (e.g., discovering candidate molecules by joining fragmented evidence). The dataset consists of Papers, Facts, and Molecules. Benchmark and workload details are available in our technical report (https://anonymous.4open.science/r/DASE-2C61/).

Additionally, we evaluated DASE on **SemBench** [18] to explore its effectiveness in prefiltering for semantic queries.

Workloads. We developed synthetic workloads over our two datasets, following a taxonomy shown in Table 6 and fully detailed in our technical report. These cover all possible combinations of structured predicates ($\mathcal{P}$), multi-component scoring ($\mathcal{S}$), and cross-table joins ($\mathcal{J}$). In this framework, single-vector similarity search serves as the default configuration. The component $\mathcal{S}$ represents the transition from simple embedding retrieval to a generalized scoring function $f(s_1, \ldots, s_n)$. This allows the system to aggregate multiple vector similarities, as required for complex discovery workflows.

We also execute the workload of the SemBench benchmark, performing the prefiltering computation in DASE and doing final semantic reasoning using Google BigQuery.

Implementation strategies. In deep research tasks, we compare DASE against five distinct execution strategies and DBMSs:

(1) **PostgreSQL:** The multi-step reasoning query is submitted directly to the PostgreSQL engine. The embedding columns are indexed using `pgvector`'s HNSW or IVFFlat structures. All execution decisions, including join order, index selection, and predicate pushdown, are delegated to the PostgreSQL optimizer.
(2) **Filter-First:** Evaluates structural and exact-match predicates first (using the `WHERE` clause) to generate a candidate set of valid IDs. The system then performs random access to fetch the corresponding embeddings, computes the similarity scores on-the-fly, and maintains a Top-k heap to produce the result.
(3) **Score-First:** An inverted strategy that prioritizes similarity search. It uses an `ORDER BY` clause to retrieve a continuous stream of records sorted by their joint similarity score. The system applies the structural predicates as a post-filter and halts the stream immediately once k valid entities are identified.
(4) **Rerank:** For queries involving multiple scoring signals, executes a retrieve-and-rerank pipeline. It streams candidates separately from each signal's index, applies predicates as post-filters, and dynamically evaluates the joint score to update the Top-k results until the exploration stopping criteria are met. For single-signal queries, it gracefully degrades to Score-First.
(5) **Milvus:** Milvus [44] is a state-of-the-art vector database. For single-signal queries (with or without structural predicates), we push down the filters directly into Milvus. Because Milvus lacks support for multi-signal scoring or relational joins, queries involving these operations are streamed to the application layer, which applies the Rerank strategy to finalize the Top-k results.

5.1 Setup and Methodology

Experiments were conducted on PostgreSQL 15.4 with the `pgvector` extension (v0.5.1) enabled. We evaluated a set of hybrid similarity queries involving structured filters, multi-vector relevance scoring, and join-based conditions. The database was hosted on a Google Cloud Platform instance provisioned with 4 vCPUs (Intel Xeon Platinum 8481C @ 2.70GHz) and 16GB of RAM. This hardware natively supports the AVX-512 instruction set, which `pgvector` leverages to accelerate vector distance computations. Evaluation was CPU-based; no GPUs were used. Each workload is executed 5 times, and we report the average query-per-second (QPS). Index construction time is isolated and excluded from all QPS measurements. All configuration parameters were left at their default values.

For the SemBench evaluation, we run two configurations. **DASE** runs ranked DASE alone, using embedding-distance-based filtering. Semantic filters are resolved via counterfactual prompts (e.g., "this is X" versus "this is not X") and semantic joins via a calibrated distance threshold. **DASE + BigQuery** then demonstrates DASE's role as a **prefilter** for a semantic operator system: DASE produces a

Table 2: Performance Comparison: Recall and QPS for Multi-Step Reasoning Queries

	PostgreSQL		Filter-First		Score-First		Rerank		Milvus		DASE	
Workload	**Recall**	**QPS**	**Recall**	**QPS**	**Recall**	**QPS**	**Recall**	**QPS**	**Recall**	**QPS**	**Recall**	**QPS**
(a) IMDB Scenario												
W1	0.96	317.7	**1.00**	3.4	0.96	317.7	0.96	317.7	0.90	**346.0**	0.96	317.7
W2	**1.00**	9.3	**1.00**	7.8	0.93	2.6	0.93	2.6	0.95	**223.9**	0.99	66.6
W3	**1.00**	1.2	**1.00**	1.2	**1.00**	1.2	0.97	30.6	0.99	8.5	0.99	**36.3**
W4	**1.00**	2.9	**1.00**	3.0	**1.00**	1.6	0.92	2.0	0.98	6.8	0.97	**8.2**
W5	**0.97**	1.2	0.94	0.09	**0.97**	1.2	**0.97**	1.2	**0.97**	4.4	0.96	**49.8**
W6	**1.00**	0.01	**1.00**	0.04	0.88	0.09	0.88	0.09	0.95	1.6	0.99	**9.8**
W7	0.03	0.01	0.03	0.01	N/A	N/A	0.04	0.03	0.20	0.07	**0.96**	**5.1**
W8	0.37	0.01	0.41	0.01	N/A	N/A	0.26	0.02	0.52	0.12	**0.98**	**2.3**
(b) MOLECULE Scenario												
W1	0.96	**366.3**	**1.00**	6.2	0.96	**366.3**	0.96	**366.3**	0.97	360.6	0.96	**366.3**
W2	**1.00**	13.3	**1.00**	9.0	0.95	8.2	0.95	8.2	0.97	**270.7**	0.98	36.4
W3	**1.00**	4.4	**1.00**	4.4	**1.00**	4.4	0.99	28.1	0.95	14.8	0.99	**30.9**
W4	**1.00**	10.6	**1.00**	9.0	**1.00**	3.0	0.98	6.3	**1.00**	**21.3**	0.99	17.7
W5	**0.99**	0.87	0.43	0.05	**0.99**	0.87	**0.99**	0.87	**0.99**	2.8	**0.99**	**129.8**
W6	0.22	0.04	0.31	0.03	0.39	0.04	0.39	0.04	0.91	0.08	**0.99**	**2.0**
W7	0.01	0.03	0.01	0.03	N/A	N/A	0.18	0.04	0.25	0.07	**0.98**	**0.77**
W8	0.12	0.04	0.11	0.03	N/A	N/A	0.21	0.02	0.56	0.12	**0.99**	**0.77**

high-recall candidate set, which BigQuery then evaluates with LLM-based semantic operators (mostly `sem_filters`). Text embeddings are generated via `Gemini-embedding-001`; multi-modal data is first described by `Gemini-Flash-2.5` and then embedded.

5.2 Multi-Step Top-k Reasoning

Deep Research. We evaluate DASE and its alternatives across the **IMDB** and **MOLECULE** datasets over our 8 workloads, reporting recall and query throughput (QPS) in Table 2. Queries exceeding a 30-second timeout are terminated and scored on results returned. Starting with the simple workloads in both scenarios, we observe that W1 is very fast for strategies that leverage the HNSW index. Milvus excels at single-signal queries. Its advantage is reversed in the workloads that require multi-score signal computation and ranking. Several other strategies revert to simpler implementations for certain workloads: for queries without a predicate, filter-first degenerates to sequential scan; with a single signal, rerank falls back to score-first. Workload W3 shows that adaptive stream selection in the threshold algorithm is helpful, as DASE outperforms the rerank strategy that also leverages the TA under a round-robin policy. For the multi-signal, multi-step workloads (W5–W8), DASE and its underlying SemJI strategy provide significant speedups over the alternative strategies. The *Score-first* strategy proved entirely unfeasible for W_7 and W_8, as it computes a full Cartesian product.

Semantic queries. Table 3 summarizes cost, quality, and latency on SemBench. DASE alone suffices for semantic-filter (**F**) tasks that reduce to a binary or coarse distinction, such as positive vs. negative reviews in *Movie*: comparing embedding distance to counterfactual anchors returns valid answers at orders-of-magnitude lower cost and latency than any LLM-backed baseline. For semantic-merge (**M**) and semantic-join (**J**) tasks that exceed embedding-distance reasoning, DASE + BigQuery substantially raises quality (e.g., *MMQA* avg. 0.11 → 0.76). Prefiltering with DASE also lets BigQuery beat its own standalone numbers at a fraction of the cost: on *E-Commerce*, \$0.54/38s at quality 0.80 versus BigQuery alone at \$2.42/45s and 0.67. The LLM sees only the prefilter's survivors, so it spends less and decides better. The one failure mode is embedding quality itself: in *Wildlife*, rare words like "impala" are poorly aligned, so neither configuration recovers fully.

Overall, DASE delivers low-cost answers when embedding distance suffices, and a high-recall prefilter that makes downstream LLM evaluation more efficient and accurate when it does not.

5.3 SemJI Construction and Maintenance

Creation. We build SemJI using HNSW-based ANN range queries with a radius threshold. Table 4 shows an ablation from a naive iterative approach, through a batched lateral join, to a fully vectorized whole-table HNSW range query — a single `JOIN LATERAL` that runs entirely in-database. This reduces construction time from 81.2 hours to 1.6 minutes, only losing 2% from recall.

SemJI evolution. We next study how well SemJI adapts its indexing threshold to the workload. To evaluate this, we reduced the MOLECULE W5 query to only evaluate linkage, with 2,000 seed queries. We initialize τ_{init} using a random warmup sample of data-to-data pairs, establishing a starting threshold ($\tau_{init} = 0.607$) before any workload biases are introduced.

Table 3: SemBench cost, quality, and latency vs. alternative systems

	LOTUS [31]			Palimpzest [23]			ThalamusDB [14]			BigQuery [9]			DASE			DASE + BigQuery		
	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.	Cost	Qual.	Lat.
(a) Movie Scenario																		
Preprocess	X	X	X	X	X	X	X	X	X	X	X	X	$3·10^{-3}$	X	27.6 s	$3·10^{-3}$	X	27.6 s
Q1: F L	$0.09	**1.00**	33.1 s	$1·10^{-3}$	**1.00**	3.8 s	$4·10^{-4}$	0.95	4.2 s	$0.05	**1.00**	26.3 s	$2.4·10^{-6}$	**1.00**	0.5 s	$1·10^{-4}$	**1.00**	3.8 s
Q2: F L	$0.01	**1.00**	2.1 s	$0.01	**1.00**	29.7 s	$2·10^{-3}$	0.92	1.9 s	$3·10^{-3}$	**1.00**	9.5 s	$2.4·10^{-6}$	**1.00**	0.5 s	$1·10^{-4}$	**1.00**	4.2 s
Q3: F	$5·10^{-3}$	0.64	2.1 s	$0.01	0.64	4.6 s	$2·10^{-3}$	0.74	3.1 s	$3·10^{-3}$	0.64	11.0 s	$3.6·10^{-6}$	**0.82**	0.5 s	$7·10^{-4}$	**0.82**	6.2 s
Q4: F	$5·10^{-3}$	0.64	2.8 s	$0.02	0.74	4.4 s	$2·10^{-3}$	0.74	3.8 s	$3·10^{-3}$	0.64	11.4 s	$3.6·10^{-6}$	**0.82**	0.5 s	$7·10^{-4}$	**0.82**	5.3 s
Q5: J L	$2.38	0.59	536.5 s	$0.01	0.39	1.9 s	$1·10^{-3}$	**1.00**	2.3 s	$1.01	0.89	54.5 s	$3.6·10^{-6}$	**1.00**	0.5 s	$6·10^{-4}$	0.82	3.9 s
Q6: J L	$1.81	0.67	432.4 s	$0.01	0.83	2.3 s	$9·10^{-4}$	0.84	1.7 s	$1.00	0.69	54.5 s	$3.6·10^{-6}$	**1.00**	0.5 s	$7·10^{-4}$	0.94	3.8 s
Q7: J	$1.81	0.21	431.8 s	$7.72	0.68	1056.1 s	$0.15	0.57	636.9 s	$3.31	0.70	198.3 s	$3.6·10^{-6}$	**0.73**	0.5 s	$9·10^{-4}$	0.71	3.8 s
Q8: C	$4·10^{-3}$	**0.93**	2.3 s	$0.02	0.86	4.3 s	$5·10^{-3}$	0.83	6.8 s	$3·10^{-3}$	0.76	10.9 s	$3.6·10^{-6}$	0.89	0.5 s	$7·10^{-4}$	0.89	4.2 s
Q9: R	$0.02	0.75	4.9 s	$0.05	**0.78**	5.7 s	X	X	X	$0.02	**0.78**	13.3 s	$4.8·10^{-6}$	0.66	0.5 s	$6·10^{-3}$	0.74	12.8 s
Q10: R	$0.13	0.40	30.9 s	$0.38	0.42	39.2 s	X	X	X	$0.13	**0.44**	32.1 s	$4.8·10^{-6}$	0.43	0.5 s	$0.085	0.41	26.1 s
Avg	$0.626	0.68	147.9 s	$0.823	0.73	115.2 s	$0.02	0.82	82.6 s	$0.553	0.75	42.2 s	$3·10^{-4}$	**0.83**	3.0 s	$9·10^{-3}$	0.81	9.2 s
(b) E-Commerce Scenario																		
Preprocess	X	X	X	X	X	X	X	X	X	X	X	X	$0.03	X	124.0 s	$0.03	X	124.0 s
Q1: F	$0.06	**1.00**	12.2 s	$0.08	**1.00**	12.2 s	$0.03	**1.00**	34.8 s	$0.04	0.59	21.2 s	$5·10^{-6}$	0.94	0.7 s	$7·10^{-3}$	**1.00**	8.0 s
Q2: F	$0.18	**0.87**	166.1 s	$0.38	0.83	57.5 s	$0.31	0.67	100.8 s	$3.96	0.21	55.7 s	$7·10^{-6}$	0.47	0.7 s	$0.02	0.66	5.5 s
Q3: M	$0.07	0.97	17.1 s	$0.12	**0.98**	16.5 s	X	X	X	$0.12	0.97	21.2 s	X	X	X	$0.04	0.93	10.5 s
Q4: M	$0.14	0.45	156.7 s	$0.24	0.53	335.0 s	X	X	X	$0.37	**0.69**	31.0 s	X	X	X	$0.17	0.67	18.3 s
Q5: C	$0.04	**0.99**	7.4 s	$0.07	0.98	6.6 s	X	X	X	$0.17	0.98	25.7 s	$4·10^{-6}$	0.96	0.7 s	$0.01	0.98	5.8 s
Q6: C	$0.12	0.89	114.8 s	$0.43	0.89	143.7 s	X	X	X	$0.35	0.88	34.9 s	$4·10^{-6}$	0.47	0.7 s	$0.16	**0.95**	19.5 s
Q7: J	$1.33	0.75	199.4 s	$1.79	**0.92**	287.6 s	$0.08	0.51	97.7 s	$0.86	0.83	45.4 s	$1·10^{-9}$	0.78	$2·10^{-3}$ s	$0.09	0.81	7.4 s
Q8: J	$4·10^{-3}$	**1.00**	4.1 s	$0.01	**1.00**	3.9 s	$0.30	0.00	713.0 s	$18.23	0.29	126.2 s	$1·10^{-9}$	0.25	$1·10^{-3}$ s	$0.10	**1.00**	3.4 s
Q9: J	$0.06	0.55	243.4 s	$0.10	0.49	44.9 s	$0.31	0.00	872.2 s	$0.21	0.58	48.6 s	$1·10^{-9}$	0.41	$1·10^{-3}$ s	$0.07	**0.83**	18.8 s
Q10: F J	$0.21	0.00	519.5 s	$1.02	0.06	1192.5 s	X	X	X	X	X	X	$1·10^{-5}$	X	0.7 s	$0.65	**0.34**	90.1 s
Q11: F J	$0.27	**0.78**	158.7 s	$0.61	0.73	132.4 s	X	X	X	X	X	X	X	X	X	$1.68	0.40	131.3 s
Q12: F M	$0.10	0.60	36.4 s	$0.14	0.00	31.9 s	X	X	X	$0.10	**0.97**	31.1 s	$6·10^{-6}$	0.85	0.7 s	$0.08	0.95	32.1 s
Q13: F	$0.24	**0.74**	274.8 s	$0.44	**0.74**	238.3 s	X	X	X	$0.38	0.70	22.4 s	$2·10^{-5}$	0.58	1.0 s	$0.13	0.72	11.6 s
Q14: F J R	$0.23	0.87	178.2 s	X	X	X	X	X	X	$4.26	0.37	73.6 s	$5·10^{-6}$	0.00	0.7 s	$4.83	**1.00**	87.3 s
Avg	$0.22	0.74	149.2 s	$0.42	0.70	192.5 s	$0.21	0.43	363.7 s	$2.42	0.67	44.8 s	$3·10^{-3}$	0.57	10.8 s	$0.54	**0.80**	38.2 s
(c) Wildlife Scenario																		
Preprocess	X	X	X	X	X	X	X	X	X	X	X	X	$0.01	X	53.9 s	$0.01	X	53.9 s
Q1: F	$0.11	**0.79**	92.4 s	$0.13	**0.79**	32.8 s	$0.11	**0.79**	19.6 s	$0.11	**0.79**	32.0 s	$1·10^{-9}$	**0.79**	$5·10^{-4}$ s	$0.02	0.78	10.1 s
Q2: F	X	X	X	$0.01	0.17	2.8 s	$0.01	0.14	4.5 s	$0.01	0.19	9.4 s	$1·10^{-9}$	**0.83**	$5·10^{-4}$ s	$3·10^{-3}$	0.50	6.2 s
Q3: F L	$0.11	**1.00**	99.4 s	$0.13	0.00	22.7 s	$0.03	0.00	4.3 s	$0.11	0.00	25.7 s	$1·10^{-9}$	0.00	$8·10^{-4}$ s	$0.02	**1.00**	9.0 s
Q4: F L	X	X	X	$0.01	0.00	2.7 s	$1·10^{-3}$	0.00	1.3 s	$0.01	**1.00**	9.4 s	$1·10^{-9}$	**1.00**	$8·10^{-4}$ s	$3·10^{-3}$	**1.00**	6.1 s
Q5: F	X	X	X	$0.13	0.75	13.5 s	$0.01	0.75	2.3 s	$0.12	0.75	19.2 s	$1·10^{-9}$	**1.00**	$1·10^{-3}$ s	$0.02	0.85	7.0 s
Q6: F	X	X	X	$0.13	0.00	19.3 s	$0.06	**0.50**	13.2 s	$0.12	0.20	24.3 s	$1·10^{-9}$	0.00	$1·10^{-3}$ s	$0.06	0.40	13.9 s
Q7: F	$0.23	**1.00**	188.3 s	$0.13	**1.00**	43.9 s	$0.20	**1.00**	28.8 s	$0.22	**1.00**	24.6 s	$1·10^{-9}$	0.00	$1·10^{-3}$ s	$0.14	**1.00**	19.9 s
Q8: F	X	X	X	$0.13	**0.75**	34.9 s	$0.12	**0.75**	23.8 s	$0.23	**0.75**	35.5 s	$1·10^{-9}$	0.00	$2·10^{-3}$ s	$0.16	0.74	24.8 s
Q9: F	X	X	X	$0.13	0.57	19.1 s	$0.08	**0.67**	17.2 s	$0.12	0.59	37.6 s	$1·10^{-9}$	0.00	$1·10^{-3}$ s	$0.06	0.40	19.5 s
Q10: F L	$0.11	**1.00**	87.8 s	$0.13	0.00	19.6 s	$0.03	0.00	4.4 s	$0.11	0.00	40.3 s	$1·10^{-9}$	**1.00**	$8·10^{-4}$ s	$0.06	0.25	24.2 s
Avg	$0.14	**0.94**	117.0 s	$0.11	0.40	21.1 s	$0.07	0.46	11.9 s	$0.12	0.52	25.8 s	$9·10^{-4}$	0.46	4.9 s	$0.05	0.69	17.7 s
(d) MMQA Scenario																		
Preprocess	X	X	X	X	X	X	X	X	X	X	X	X	$0.01	X	50.1 s	$0.01	X	50.1 s
Q1: M	$0.02	**1.00**	7.4 s	$3·10^{-3}$	**1.00**	4.5 s	X	X	X	$0.01	**1.00**	14.1 s	X	X	X	$0.01	**1.00**	14.1 s
Q2a: J	$0.89	0.83	169.6 s	$1.04	**1.00**	135.9 s	X	X	X	$0.08	0.00	53.8 s	$1·10^{-5}$	0.08	1.0 s	$0.04	0.57	28.6 s
Q2b: J	$0.89	0.83	166.8 s	$1.04	**1.00**	133.8 s	X	X	X	$0.12	0.00	38.4 s	$1·10^{-5}$	0.08	1.0 s	$0.06	0.57	21.4 s
Q3a: F	$0.01	**0.83**	4.2 s	$0.02	0.80	4.5 s	$0.01	0.75	11.0 s	$0.01	0.72	19.6 s	$5·10^{-6}$	0.38	0.7 s	$9·10^{-3}$	0.72	20.5 s
Q3f: F	$0.01	**1.00**	4.2 s	$0.02	**1.00**	8.0 s	$0.01	**1.00**	7.0 s	$0.01	0.67	22.3 s	$5·10^{-6}$	0.09	0.7 s	$0.01	0.67	22.3 s
Q4: M	X	X	X	$5·10^{-3}$	0.54	1.2 s	X	X	X	$2·10^{-3}$	0.60	9.7 s	X	X	X	$2·10^{-3}$	**0.61**	9.5 s
Q5: M	$1·10^{-3}$	**1.00**	0.48 s	$1·10^{-3}$	**1.00**	0.50 s	X	X	X	X	X	X	X	X	X	X	X	X
Q6a: F	$0.01	**1.00**	4.4 s	$0.02	**1.00**	4.0 s	$2·10^{-3}$	0.33	5.6 s	$4·10^{-3}$	0.03	18.9 s	$5·10^{-6}$	0.04	0.7 s	$3·10^{-3}$	**1.00**	15.8 s
Q6b: F	$0.01	**1.00**	3.8 s	$0.02	**1.00**	4.1 s	$2·10^{-3}$	**1.00**	5.5 s	$4·10^{-3}$	0.04	17.3 s	$5·10^{-6}$	0.01	0.7 s	$4·10^{-3}$	0.87	20.4 s
Q6c: F	$0.01	**1.00**	4.6 s	$0.02	**1.00**	4.7 s	$2·10^{-3}$	0.53	13.4 s	$4·10^{-3}$	0.13	17.4 s	$5·10^{-6}$	0.10	0.7 s	$4·10^{-3}$	0.66	19.1 s
Q7: J	$13.61	0.32	2311.4 s	$15.65	0.31	2101.6 s	X	X	X	$1.18	0.00	91.7 s	$1·10^{-5}$	0.10	0.7 s	$0.04	**1.00**	6.9 s
Avg	$1.55	**0.88**	267.7 s	$1.62	0.87	218.4 s	$5·10^{-3}$	0.72	8.5 s	$0.14	0.31	30.3 s	$1·10^{-3}$	0.11	6.3 s	$0.02	0.76	20.8 s
(e) Cars Scenario																		
Preprocess	X	X	X	X	X	X	X	X	X	X	X	X	$2.50	X	1300 s	$2.50	X	1300 s
Q1: F	$1.74	**0.90**	550.0 s	$2.44	0.69	465.6 s	$1.37	0.81	829.7 s	$1.44	0.71	61.7 s	$5·10^{-6}$	0.66	0.9 s	$0.31	0.78	22.0 s
Q2: F	X	X	X	$4·10^{-3}$	0.00	4.1 s	$0.01	0.09	14.1 s	$0.01	0.08	14.1 s	$5·10^{-6}$	0.00	0.7 s	$1·10^{-3}$	**0.16**	5.1 s
Q3: F L	$0.60	0.90	456.2 s	$0.01	0.92	6.1 s	X	X	X	$1.66	**1.00**	36.4 s	$5·10^{-6}$	**1.00**	0.7 s	$2·10^{-3}$	**1.00**	4.3 s
Q4: F	$1.71	0.99	822.0 s	$2.41	0.99	443.6 s	$1.34	**1.00**	768.8 s	$1.41	0.99	68.7 s	$5·10^{-6}$	0.99	0.9 s	$0.31	0.99	23.4 s
Q5: F	X	X	X	$0.01	**1.00**	6.3 s	$1.61	**1.00**	483.7 s	$1.47	**1.00**	58.9 s	$1·10^{-5}$	0.50	1.1 s	$7·10^{-3}$	**1.00**	5.1 s
Q6: F J	X	X	X	$2.51	0.96	427.3 s	$1.96	**0.97**	775.4 s	$2.00	0.96	44.3 s	$1·10^{-5}$	0.88	1.3 s	$0.89	0.96	41.2 s
Q7: F	X	X	X	$4.47	0.56	882.7 s	$3.06	**0.58**	2146.2 s	$3.17	0.45	86.0 s	$1·10^{-5}$	0.47	1.3 s	$1.55	0.52	131.5 s
Q8: F L	$1.78	**0.45**	1349.8 s	$2.02	0.29	268.7 s	$0.21	0.20	68.0 s	$1.69	0.24	38.2 s	$5·10^{-6}$	0.15	0.7 s	$0.04	0.39	6.0 s
Q9: F	X	X	X	$2.05	0.00	308.1 s	$1.61	**1.00**	335.8 s	$0.03	0.00	13.6 s	$5·10^{-6}$	**1.00**	0.7 s	$0.03	0.00	16.4 s
Q10: C	$3.09	0.41	618.1 s	$4.69	0.51	594.9 s	X	X	X	$2.70	**0.57**	62.0 s	$2·10^{-5}$	0.45	1.8 s	$0.86	0.55	27.6 s
Avg	$1.78	**0.73**	759.2 s	$2.06	0.59	340.7 s	$1.40	0.70	677.7 s	$1.56	0.60	48.4 s	$0.23	0.61	119.1 s	$0.59	0.63	143.9 s

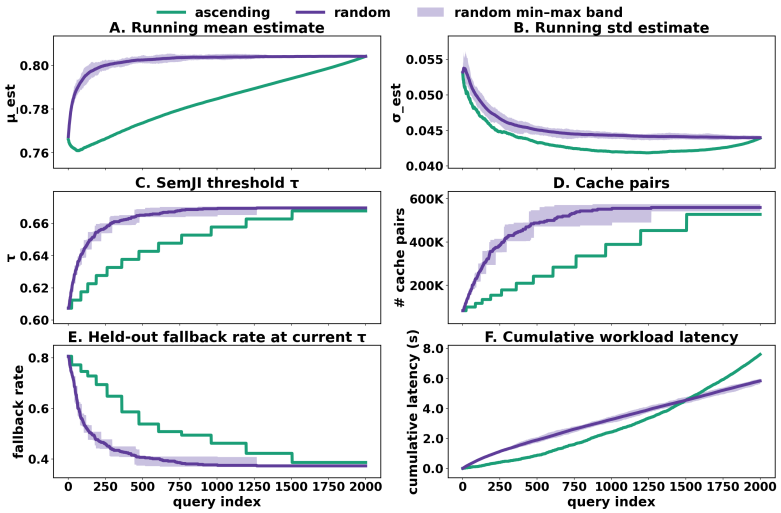

Figure 5: SemJI evolution test.

Method	Recall	Time
Exact Construction	1.00	~81.2 h
Iterative	0.98	14.0 m
Batched	0.98	3.0 m
Fully Vectorized	0.98	1.6 m

Table 4: SemJI Construction

Method	Setting	Recall	QPS
baseline	-	1.0	0.23
Bitmap	2-hop on	0.905	9.6
	2-hop off	0.884	10.5
	adaptive	0.903	10.1
Bloom	bits/element = 4	0.817	9.1
	bits/element = 12	0.872	8.8
	bits/element = 20	0.883	8.2

Table 5: Filtered search results.

We then tracked the online evolution of the distance threshold τ (Figure 5). To test robustness against workload drift, we compared two query arrival patterns: *ascending order* (distribution shift: earlier queries have more near-neighbors than later ones) and *random order* (stationary distribution). Regardless of arrival order, the online Welford estimator converges μ and σ to a nearly identical final state (Figure 5.A/B), and τ converges to within 0.003 of the same value $\tau_{converged} \approx 0.665$ (Figure 5.C), after 12 radius expansions in either ordering. The materialized cache likewise reaches comparable size ($\approx$ 550K pairs, Figure 5.D). Arrival order alters the adaptation *trajectory*, but not the final index topology. Online adaptation dropped the held-out fallback rate from an initial 80.6% down to a permanent floor of roughly 37% at $\tau_{converged}$ (Figure 5.E).

The cost of biased arrival appears only late, and for a subtle reason. Ascending order places the most demanding queries—those requesting the largest search radii—at the end, while the small-radius early queries grow τ and the cache only slowly. The demanding tail thus arrives when the cache has been under-grown the longest, yielding a 1.30× cumulative latency penalty at 2,000 queries (Figure 5.F). The adaptation strategy is thus robust: arrival order shifts when the cost is paid but does not compromise the final index, and the worst-case penalty is bounded.

5.4 Filter Sets in the ANN Index

Table 5 evaluates predicate-aware HNSW search strategies. The exact baseline guarantees perfect recall but at low throughput (0.23 QPS). To accelerate filtering, we compare Bitmap and Bloom filter structures, finding that the Bitmap approach consistently delivers better recall and higher QPS across all configurations. Within the Bitmap approach, we further evaluate graph traversal methods. The 2-hop adaptive strategy yields the optimal trade-off: by dynamically exploring extended neighbors only when valid nodes are sparse, it achieves a roughly 40× speedup (10.1 QPS) over the baseline while maintaining a highly competitive recall of 0.903. This adaptive mechanism successfully balances the speed of the "2-hop off" method with the improved accuracy of the "always-on" method.

6 Related Work

Our task of multi-step reasoning spans "deep research" [12, 50], "wide search" tools [17, 47], LLM-based information extraction [1, 38], and semantic operator systems [23, 31]. We focus on the storage-and-execution layer: efficiently retrieving, filtering, and joining disparate objects to form unified answers. Neither traditional vector DBMSs [34, 45], hybrid systems like Milvus [44], nor relational DBMS hybrid query work [11, 24] provide sufficient expressiveness for this setting, and IR-style reranking [27] is insufficiently efficient.

We model our problem as one of joining multiple streams using threshold-based top-k evaluation strategies [8, 16, 41]. However, such methods require bounds on scoring features, which is non-trivial for embedding-similarity queries; our indexing strategies address this directly. These thresholds prune candidates before costlier semantic operator evaluation [29].

7 Conclusions

Multi-step reasoning queries frequently arise in agentic data engineering tasks such as scientific discovery. We introduced SemJI, which facilitates efficient candidate exploration in top-k multi-step reasoning queries, along with ANN extensions to enable batched requests. We developed DASE, a query processing strategy that jointly optimizes attribute predicates and vector similarity scoring across relational joins. Finally, we conducted a comprehensive evaluation on existing semantic benchmarks as well as a novel benchmark specifically designed to capture the structure of multi-step reasoning queries in scientific discovery settings. Our results validate the effectiveness of SemJI.

We hope that our formalization, methods, and benchmarks serve as a foundation for continued progress on query systems that bridge structured and unstructured data at scale. In future work, we continue to develop a broader agentic query planner which delegates computational tasks to our DASE engine.

References

[1] Eric Anderson, Jonathan Fritz, Austin Lee, Bohou Li, Mark Lindblad, Henry Lindeman, Alex Meyer, Parth Parmar, Tanvi Ranade, Mehul A. Shah, Ben Sowell, Dan G. Tecuci, Vinayak Thapliyal, and Matt Welsh. 2024. The Design of an LLM-powered Unstructured Analytics System. *ArXiv* abs/2409.00847 (2024). https://api.semanticscholar.org/CorpusID:272368251

[2] Burton H. Bloom. 1970. Space/Time Trade-offs in Hash Coding with Allowable Errors. *CACM* 13, 7 (July 1970), 422–426.

[3] Huiping Cao, Yan Qi, K Selçuk Candan, and Maria Luisa Sapino. 2010. Feedback-driven result ranking and query refinement for exploring semi-structured data collections. In *EDBT*. 3–14.

[4] Mingju Chen, Guibin Zhang, Heng Chang, Yuchen Guo, and Shiji Zhou. 2026. A-MapReduce: Executing Wide Search via Agentic MapReduce. arXiv:2602.01331 [cs.MA] https://arxiv.org/abs/2602.01331

[5] Xu Chen, Zheng Qin, Yongfeng Zhang, and Tao Xu. 2016. Learning to rank features for recommendation over multiple categories. In *Proceedings of the 39th International ACM SIGIR conference on Research and Development in Information Retrieval*. 305–314.

[6] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. doi:10.48550/arXiv.2504.19413 arXiv:2504.19413.

[7] Seyone Chithrananda, Gabriel Grand, and Bharath Ramsundar. 2020. ChemBERTa: Large-Scale Self-Supervised Pretraining for Molecular Property Prediction. doi:10.48550/arXiv.2010.09885 arXiv:2010.09885.

[8] Ronald Fagin, Amnon Lotem, and Moni Naor. 2003. Optimal aggregation algorithms for middleware. *Journal of computer and system sciences* 66, 4 (2003), 614–656.

[9] Sérgio Fernandes and Jorge Bernardino. 2015. What is bigquery?. In *Proceedings of the 19th International Database Engineering & Applications Symposium*. 202–203.

[10] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Qianyu Guo, Meng Wang, and Haofen Wang. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *ArXiv* abs/2312.10997 (2023). https://api.semanticscholar.org/CorpusID:266359151

[11] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, and others. 2023. Filtered-diskann: Graph algorithms for approximate nearest neighbor search with filters. In *Proceedings of the ACM Web Conference 2023*. 3406–3416.

[12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. 2025. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* 645, 8081 (Sept. 2025), 633–638. doi:10.1038/s41586-025-09422-z

[13] Ziyang Huang, Haolin Ren, Xiaowei Yuan, Jiawei Wang, Zhongtao Jiang, Kun Xu, Shizhu He, Jun Zhao, and Kang Liu. 2026. WideSeek: Advancing Wide Research via Multi-Agent Scaling. *arXiv preprint arXiv:2602.02636* (2026).

[14] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proceedings of the ACM on Management of Data* 2, 3 (May 2024), 1–26. doi:10.1145/3654989

[15] Haydn Thomas Jones, Natalie Maus, Josh magnus Ludan, Maggie Ziyu Huan, Jiaming Liang, Marcelo Der Torossian Torres, Jiatao Liang, Zachary Ives, Yoseph Barash, Cesar de la Fuente-Nunez, Jacob R. Gardner, and Mark Yatskar. 2025. A Dataset for Distilling Knowledge Priors from Literature for Scientific Discovery. (May 2025).

[16] Benny Kimelfeld and Yehoshua Sagiv. 2006. Finding and approximating top-k answers in keyword proximity search. In *PODS*. 173–182.

[17] Tian Lan, Bin Zhu, Qianghuai Jia, Junyang Ren, Haijun Li, Longyue Wang, Zhao Xu, Weihua Luo, and Kaifu Zhang. 2025. Deepwidesearch: Benchmarking depth and width in agentic information seeking. *arXiv preprint arXiv:2510.20168* (2025).

[18] Jiale Lao, Andreas Zimmerer, Olga Ovcharenko, Tianji Cong, Matthew Russo, Gerardo Vitagliano, Michael Cochez, Fatma Özcan, Gautam Gupta, Thibaud Hottelier, H. V. Jagadish, Kris Kissel, Sebastian Schelter, Andreas Kipf, and Immanuel Trummer. 2025. SemBench: A Benchmark for Semantic Query Processing Engines. doi:10.48550/arXiv.2511.01716 arXiv:2511.01716.

[19] Jinhyuk Lee, Feiyang Chen, Sahil Dua, Daniel Cer, et al. 2025. Gemini Embedding: Generalizable Embeddings from Gemini. *Technical Report, Google DeepMind* (2025). https://arxiv.org/abs/2412.03511

[20] Chengkai Li, Kevin Chen-Chuan Chang, Ihab F. Ilyas, and Sumin Song. 2005. RankSQL: Query Algebra and Optimization for Relational Top-k Queries. In *SIGMOD*. 131–142.

[21] Hang Li. 2011. *Learning to Rank for Information Retrieval and Natural Language Processing*. Morgan Claypool.

[22] Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhen Tao, Huayi Lai, Hao Wu, Bo Tang, Zhengren Wang, Zhaoxin Fan, Ningyu Zhang, Linfeng Zhang, Junchi Yan, Mingchuan Yang, Tong Xu, Wei Xu, Huajun Chen, Haofen Wang, Hongkang Yang, Wentao Zhang, Zhi-Qin John Xu, Siheng Chen, and Feiyu Xiong. 2025. MemOS: A Memory OS for AI System. doi:10.48550/arXiv.2507.03724 arXiv:2507.03724.

[23] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpzest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*. 2.

[24] Duo Lu, Helena Caminal, Manos Chatzakis, Yannis Papakonstantinou, Yannis Chronis, Vaibhav Jain, and Fatma Özcan. 2026. An In-Depth Study of Filter-Agnostic Vector Search on a PostgreSQL Database System: [Experiments and Analysis]. doi:10.48550/arXiv.2603.23710 arXiv:2603.23710.

[25] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. 2025. A Survey of Context Engineering for Large Language Models. doi:10.48550/arXiv.2507.13334 arXiv:2507.13334.

[26] Fatemeh Nargesian, Erkang Zhu, Renée J. Miller, Ken Q. Pu, and Patricia C. Arocena. 2019. Data Lake Management: Challenges and Opportunities. *Proc. VLDB Endow.* 12, 12 (2019), 1986–1989. doi:10.14778/3352063.3352116

[27] Rodrigo Nogueira and Kyunghyun Cho. 2020. Passage Re-ranking with BERT. doi:10.48550/arXiv.1901.04085 arXiv:1901.04085.

[28] OpenAI. 2024. New embedding models and API updates. https://openai.com/blog/new-embedding-models-and-api-updates. Accessed: 2026-04-11.

[29] Liana Patel, Siddharth Jha, Parth Asawa, Melissa Pan, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Analytics Over Text Data. https://api.semanticscholar.org/CorpusID:271218837

[30] Liana Patel, Siddharth Jha, Parth Asawa, Melissa Pan, Carlos Guestrin, and Matei Zaharia. 2024. Semantic Operators: A Declarative Model for Rich, AI-based Analytics Over Text Data. https://api.semanticscholar.org/CorpusID:271218837

[31] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184.

[32] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. Acorn: Performant and predicate-agnostic search over vector embeddings and structured data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27.

[33] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2024. Graph Retrieval-Augmented Generation: A Survey. doi:10.48550/arXiv.2408.08921 arXiv:2408.08921.

[34] Pinecone Systems. 2026. Pinecone: The Vector Database for AI Applications. https://www.pinecone.io/. Accessed: 2026-04-15.

[35] David Rogers and Mathew Hahn. 2010. Extended-Connectivity Fingerprints. *Journal of Chemical Information and Modeling* 50, 5 (May 2010), 742–754. doi:10.1021/ci100050t

[36] Matthew Russo, Sivaprasad Sudhir, Gerardo Vitagliano, Chunwei Liu, Tim Kraska, Samuel Madden, and Michael Cafarella. 2025. Abacus: A Cost-Based Optimizer for Semantic Operator Systems. *arXiv preprint arXiv:2505.14661* (2025).

[37] Ahmmad OM Saleh, Gokhan Tur, and Yucel Saygin. 2024. SG-RAG: Multi-hop question answering with large language models through knowledge graphs. In *Proceedings of the 7th International Conference on Natural Language and Speech*

Processing (ICNLSP 2024). 439–448.
[38] Shreya Shankar, Tristan Chambers, Tarak Shah, Aditya G Parameswaran, and Eugene Wu. 2024. DocETL: Agentic Query Rewriting and Evaluation for Complex Document Processing. *arXiv preprint arXiv:2410.12189* (2024).
[39] R. Souvenir and R. Pless. 2005. Manifold clustering. In *Tenth IEEE International Conference on Computer Vision (ICCV'05) Volume 1*, Vol. 1. 648–653 Vol. 1. doi:10.1109/ICCV.2005.149 ISSN: 2380-7504.
[40] Immanuel Trummer. 2025. Implementing Semantic Join Operators Efficiently. doi:10.48550/arXiv.2510.08489 arXiv:2510.08489.
[41] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2024. Ranked Enumeration for Database Queries. *ACM SIGMOD Record* 53, 3 (Nov. 2024), 6–19. doi:10.1145/3703922.3703924
[42] Patrick Valduriez. 1987. Join Indices. *TODS* 12, 2 (1987), 218–246.
[43] Roman Vershynin. 2018. *High-dimensional probability: An introduction with applications in data science*. Vol. 47. Cambridge university press.
[44] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data*. ACM, Virtual Event China, 2614–2627. doi:10.1145/3448016.3457550
[45] Weaviate. 2026. Weaviate: An Open-Source Cloud-Native Vector Database. https://github.com/weaviate/weaviate. Accessed: 2026-04-15.
[46] David Weininger. 1988. SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of chemical information and computer sciences* 28, 1 (1988), 31–36.
[47] Ryan Wong, Jiawei Wang, Junjie Zhao, Li Chen, Yan Gao, Long Zhang, Xuan Zhou, Zuo Wang, Kai Xiang, Ge Zhang, Wenhao Huang, Yang Wang, and Ke Wang. 2025. WideSearch: Benchmarking Agentic Broad Info-Seeking. arXiv:2508.07999 [cs.CL] https://arxiv.org/abs/2508.07999
[48] Zelai Xu, Zhexuan Xu, Ruize Zhang, Chunyang Zhu, Shi Yu, Weilin Liu, Quanlu Zhang, Wenbo Ding, Chao Yu, and Yu Wang. 2026. WideSeek-R1: Exploring Width Scaling for Broad Information Seeking via Multi-Agent Reinforcement Learning. arXiv:2602.04634 [cs.AI] https://arxiv.org/abs/2602.04634
[49] Zhepeng Yan, Nan Zheng, Zachary Ives, Partha Talukdar, and Cong Yu. 2013. Actively Soliciting Feedback for Query Answers in Keyword Search-Based Data Integration. *PVLDB* (2013).
[50] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations* abs/2210.03629 (2022).

8 Benchmark Dataset

8.1 IMDB Benchmark

8.1.1 Dataset Description. The IMDB dataset is designed to simulate a movie discovery and recommendation environment. It allows for querying films based on a combination of categorical metadata (e.g., genres, ratings, release years) and rich textual profiles. To set up a scenario facilitating cross-era movie discovery (e.g., finding modern successors to classic cinema), we partition the data into two relational tables, T_1 and T_2, representing different time periods. Each record encapsulates a comprehensive movie profile, including a synthesized crew description and high-dimensional embeddings for its plot and title.

8.1.2 Data Source and Construction. To construct this benchmark, we integrated official metadata with unstructured content:

- **Data Sourcing:** We extracted structured attributes (title, rating, crew identifiers) from the official IMDB non-commercial datasets. To ensure a high-quality corpus, we limited the scope to non-adult feature films released between 1950 and 2020. This was augmented with detailed plot descriptions sourced from the `jquigl/imdb-genres` collection.
- **Profile Construction:** We synthesized an `actor_director` textual field for each movie by joining the crew identifiers with the name basics table, resolving top-billed cast members and directors into human-readable strings. The final consolidated table, consisting of 85,615 cleaned records, was merged on the composite key of movie title and release year.
- **Vectorization / Partition:** We generated 3072-dimensional embeddings for the `title`, `plot`, and `actor_director` fields using a Gemini embedding model. Finally, the dataset was split by the median release year (2007) into table T_1 (42,378 records, pre-2007) and table T_2 (43,237 records, 2007–2020) to provide physical targets for relational semantic joins.

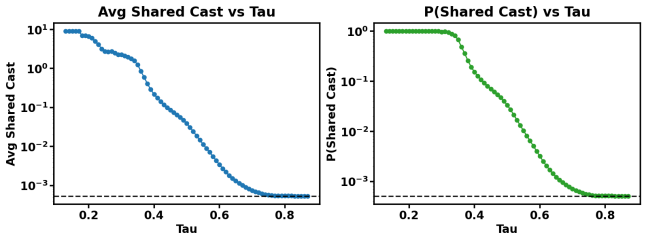

Figure 6: Validation of the semantic join proxy: shared cast metrics versus embedding distance τ. Dashed lines indicate global baselines.

8.1.3 Workload Construction. To systematically evaluate the execution strategies, we generate a synthetic workload suite that instantiates the three core dimensions of our multi-step reasoning query model: structured predicates ($\mathcal{P}$), multi-vector scoring ($\mathcal{S}$), and semantic join conditions ($\mathcal{J}$).

Predicates and Scoring ($\mathcal{P}$ and $\mathcal{S}$). We instantiate hard logical constraints ($\mathcal{P}$) using the exact relational metadata, such as bounding the release years or applying exact-match filters on categorical genres. The multi-dimensional ranking objective ($\mathcal{S}$) is formulated by aggregating the semantic similarities between user-provided query anchors and the movies' high-dimensional `plot` and `title` embeddings.

Semantic Joins ($\mathcal{J}$). To evaluate inter-entity synthesis (workloads W5–W8), we simulate a cross-era discovery scenario that links classic films in T_1 with their modern counterparts in T_2. This linkage is fundamentally driven by a semantic join over the `actor_director` embeddings.

To validate that the geometric distance (τ) in this specific embedding space serves as a robust proxy for real-world semantic relatedness, we profiled the pairwise distances across the dataset. As illustrated in Figure 6, we measured both the average number of shared cast members (left) and the probability of two movies sharing at least one cast member (right) as a function of the distance threshold τ. Note that the y-axes are scaled logarithmically, and the horizontal dashed lines denote the global expected baselines across the unconstrained Cartesian product.

The empirical results exhibit a pronounced correlation: at tight distance thresholds (small τ), both the expected density and the probability of shared cast members increase exponentially, vastly outperforming the global average. This observation perfectly aligns with our geometric premise that meaningful semantic affinities

Table 6: Example use cases of each workload on IMDB and MOLECULE.

WL	$\mathcal{P}$	Multi-$\mathcal{S}$	$\mathcal{J}$	Pattern	IMDB scenario	MOLECULE scenario
Intra-Entity Retrieval						
W_1	×	×	×	Semantic retrieval	Find movies by a plot description	Find facts on a research topic
W_2	✓	×	×	Filtered retrieval	Find movies by a plot description, within genre/era	Find facts on a research topic, within organism/property scope
W_3	×	✓	×	Multi-scoring retrieval	Find movies by plot, title, and cast all together	Find facts on a topic about a reference molecule
W_4	✓	✓	×	Filtered multi-scoring	Find movies by plot, title, and cast all together, within genre/era	Find facts on a topic about a reference molecule, within organism/property scope
Inter-Entity Synthesis						
W_5	×	×	✓	Semantic join	Pair up similar-cast films that fit a plot theme	Find paper-supported facts about a reference molecule
W_6	✓	×	✓	Filtered join	Pair up similar-cast films that fit a plot theme, within genre/era on each side	Find paper-supported facts about a reference molecule, within fact/paper filters
W_7	×	✓	✓	Multi-scoring join	Pair up similar-cast films across two plot themes	Find paper-supported facts combining a reference molecule, a topic, and tight fact-paper alignment
W_8	✓	✓	✓	Filtered multi-scoring join	Pair up similar-cast films across two plot themes, within genre/era on each side	Find paper-supported facts combining molecule/topic/fact-paper alignment, within fact/paper filters

manifest strictly as extreme tail events. Consequently, this justifies our methodology of utilizing the `actor_director` embedding distance as the underlying relational proxy to construct and evaluate ground-truth semantic join pairs.

W1: Find movies by a plot description (Semantic Retrieval). To establish a performance baseline, we synthesize a query set of 100 seed phrases simulating authentic user search behaviors. We generated these search intents using Claude Opus 4.6 (temperature = 0.8) with the following strict few-shot prompt: *"Generate exactly [N] short phrases that someone might type when searching for movies. Examples: 'thriller with twist', 'romantic comedy in Paris'. Output one phrase per line, no numbering, no other text."* Each generated phrase is then vectorized into 1536-dimensional embeddings using the Gemini-embedding-001 model. To ensure uniform coverage across the partitioned corpus, the generator randomly assigns each query to one of the four possible target spaces: either the `plot` or `title` embeddings within T_1 or T_2. Each query is parameterized with $K = 20$. This workload represents the simplest form of single-signal semantic retrieval, providing the ground-truth baseline for the system's raw vector search throughput without structural constraints.

```
-- Searching for "a thriller with unexpected twist and
    mystery"
SELECT tconst, title, plot
FROM imdb_T1
ORDER BY plot_emb <=> '[0.0336, -0.0163, ...]'
LIMIT 20;
```

Listing 2: Example W1 Query

W2: Find movies by a plot description, within genre/era (Filtered Retrieval). Building upon W1, workload W2 introduces exact structured predicates ($\mathcal{P}$) to evaluate filtered semantic search. We synthesize a query set of 100 seed phrases focusing on era-specific or genre-specific search intents. These were generated via Claude Opus 4.6 (temperature = 0.8) using the strict prompt: *"Generate exactly [N] short phrases that someone might type when searching for a specific movie era or genre. Examples: '80s action movie with cops', 'classic 60s western'. Output one phrase per line, no numbering, no other text."* The generated phrases are vectorized into 1536-dimensional embeddings using the Gemini-embedding-001 model. To construct the structured predicates, each query is assigned a random bounding interval ($[startYear, endYear]$) strictly mapped to its target table: T_1 queries are constrained to eras like [1950, 1969] or [1970, 1989], while T_2 queries are bounded by modern ranges like [2007, 2012]. Paired with a uniformly selected target space (either `plot` or `title` embeddings) and parameterized with $K = 20$, this workload benchmarks the engine's capability to efficiently execute hybrid queries combining relational range filters with dense vector scans.

```
-- Searching for "80s action movie with cops" within a
    specific era
SELECT tconst, title, startYear, plot
FROM imdb_T1
WHERE startYear >= 1970 AND startYear <= 1989
ORDER BY plot_emb <=> '[0.0152, -0.0221, ...]'
LIMIT 20;
```

Listing 3: Example W2 Query

W3: Find movies by plot, title, and cast all together (Multi-scoring Retrieval). To evaluate multi-signal aggregation (Multi-$\mathcal{S}$) without structural predicates ($\mathcal{P} = \emptyset$), workload W3 synthesizes complex search intents where a user query encompasses both thematic elements and specific crew preferences simultaneously. We generated 100 multi-faceted search profiles using Claude Opus 4.6 (temperature = 0.8). The prompt was designed to produce distinct natural language inputs for different dimensions of a movie (e.g., a specific plot description alongside a desired actor/director profile). Each distinct text field is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model. During execution, the system evaluates these distinct query vectors against their corresponding semantic spaces (e.g., `plot_emb` and `actor_director_emb`) within a randomly assigned target table (T_1 or T_2). To test the engine's capability for late-binding multi-scoring, each query is parameterized with randomized interpolation weights summing to 1.0. With $K = 20$, this workload directly isolates the performance impact of multi-stream index scanning and dynamic score aggregation.

```
-- Q: "casino heist" plot starring "Robert De Niro"
SELECT tconst, title, plot, actor_director
FROM imdb_T1
ORDER BY
  0.65 * (plot_emb <=> '[0.012, -0.043, ...]')
  -- plot intent
+ 0.35 * (actor_director_emb <=> '[-0.005, 0.081, ...]')
  -- cast intent
LIMIT 20;
```

Listing 4: Example W3 Query

W4: Find movies by plot, title, and cast all together, within genre/era (Filtered multi-scoring). Building upon W3, workload W4 introduces exact structural predicates ($\mathcal{P}$) to evaluate filtered multi-signal retrieval. We generated 100 multi-faceted, era-specific search profiles using Claude Opus 4.6 (temperature = 0.8). The prompt was designed to produce distinct natural language inputs for different dimensions of a movie (e.g., a specific plot description alongside a desired actor/director profile), while explicitly targeting a specific cinematic era. Each distinct text field is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model. To construct the structured predicates, each query is assigned a random bounding interval ($[startYear, endYear]$) strictly mapped to its target table: T_1 queries are constrained to eras like $[1950, 1969]$ or $[1970, 1989]$, while T_2 queries are bounded by modern ranges like $[2007, 2012]$. During execution, the system evaluates these distinct query vectors against their corresponding semantic spaces (e.g., `plot_emb` and `actor_director_emb`) while simultaneously enforcing the temporal filters. To test the engine's capability for late-binding filtered multi-scoring, each query is parameterized with randomized interpolation weights summing to 1.0. With $K = 20$, this workload benchmarks the performance impact of executing hybrid queries that combine relational range filters with multi-stream vector scans.

```
-- Q: "heartwarming family adventure" plot starring a "
     comedic ensemble"
SELECT tconst, title, startYear, plot, actor_director
FROM imdb_T2
WHERE startYear >= 2007 AND startYear <= 2012
ORDER BY
  0.55 * (plot_emb <=> '[0.015, -0.021, ...]')
  -- plot intent
+ 0.45 * (actor_director_emb <=> '[-0.033, 0.042, ...]')
  -- cast intent
LIMIT 20;
```

Listing 5: Example W4 Query

W5: Pair up similar-cast films that fit a plot theme (Semantic join). Transitioning to Inter-Entity Synthesis ($\mathcal{J}$), workload W5 evaluates semantic joins without structural predicates ($\mathcal{P} = \emptyset$). We synthesize 100 movie plot motifs using Claude Opus 4.6 (temperature = 0.8) with the strict prompt: *"Generate exactly [N] short phrases describing a specific movie theme, setting, or plot motif... Examples: 'a detective solving a murder in a small town', 'spaceship crew encountering an alien anomaly'. Output one phrase per line."* Each phrase is vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model. During execution, the system first retrieves the top $K = 20$ candidate movies from T_1 based on the semantic similarity of their `plot_emb` to the query intent. For each retrieved candidate, it performs a dependent K_j-NN lateral join ($K_j = 5$) against T_2, ranking results by the distance between their respective `actor_director_emb` vectors. This workload benchmarks the engine's capability to execute multi-stage semantic operations, chaining a base vector search directly into a dependent nearest-neighbor join across distinct semantic spaces.

```
-- Q: Find "detective solving a murder" plots in T1,
-- then join T2 movies with similar cast/crew
SELECT t1.title AS t1_title, t2.title AS t2_title
FROM (
  SELECT tconst, title, actor_director_emb
  FROM imdb_T1
  ORDER BY plot_emb <=> '[0.021, -0.015, ...]'
  -- plot intent
  LIMIT 20
) t1
CROSS JOIN LATERAL (
  SELECT tconst, title
  FROM imdb_T2 t2
  ORDER BY t2.actor_director_emb <=> t1.
     actor_director_emb
  -- cast similarity join
  LIMIT 5
) t2;
```

Listing 6: Example W5 Query

W6: Pair up similar-cast films that fit a plot theme, within genre/era on each side (Filtered join). Building upon W5, workload W6 introduces exact structural predicates ($\mathcal{P}$) on both sides of the semantic join to evaluate filtered inter-entity synthesis. We synthesize 100 movie plot motifs using Claude Opus 4.6 (temperature = 0.8) with the strict prompt: *"Generate exactly [N] short phrases describing a specific movie theme, setting, or plot motif. Examples: 'a detective solving a murder in a small town', 'spaceship crew encountering an alien anomaly'. Output one phrase per line, no numbering, no other text."* Each phrase is vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model. To construct the structured predicates, each query assigns a random temporal bounding interval to both the driving table and the lateral join target: T_1 queries are constrained to eras like $[1950, 1969]$ or $[1970, 1989]$,

while the dependent T_2 queries are bounded by modern ranges like [2007, 2012] or [2013, 2016]. During execution, the system first retrieves the top $K = 20$ candidate movies from T_1 satisfying both the T_1 year filter and the semantic similarity of their `plot_emb` to the query intent. For each retrieved candidate, it performs a dependent K_j-NN lateral join ($K_j = 5$) against T_2, enforcing the T_2 year filter and ranking results by the distance between their respective `actor_director_emb` vectors. This workload benchmarks the performance impact of enforcing relational filters at multiple stages of a complex semantic join pipeline.

```
-- Q: Find "post-apocalyptic survivors" plots in T1
     (1970-1989),
-- then join T2 movies (2007-2012) with similar cast/crew
SELECT t1.title AS t1_title, t2.title AS t2_title
FROM (
  SELECT tconst, title, actor_director_emb
  FROM imdb_T1
  WHERE startYear >= 1970 AND startYear <= 1989
  ORDER BY plot_emb <=> '[-0.016, -0.012, ...]'
  -- plot intent
  LIMIT 20
) t1
CROSS JOIN LATERAL (
  SELECT tconst, title
  FROM imdb_T2 t2
  WHERE t2.startYear >= 2007 AND t2.startYear <= 2012
  ORDER BY t2.actor_director_emb <=> t1.
     actor_director_emb
  -- cast similarity join
  LIMIT 5
) t2;
```

Listing 7: Example W6 Query

W7: Pair up similar-cast films across two plot themes (Multi-scoring join). Building upon W5, workload W7 introduces multi-signal aggregation (Multi-$\mathcal{S}$) into the inter-entity synthesis pipeline ($\mathcal{J}$) without structural predicates ($\mathcal{P} = \emptyset$). We synthesize 100 pairs of distinct movie concepts using Claude Opus 4.6 (temperature = 0.8) with the strict prompt: *"Generate exactly [N] pairs of movie concepts. Each line should contain two distinct concepts separated by a pipe character '|'. For example: 'A gritty sci-fi thriller about time travel | A lighthearted comedy about a robot learning to love'. Output only the pipe-separated pairs, one pair per line."* Each text field in the pair is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model. During execution, the system first retrieves the top $K = 20$ candidate movies from T_1 based on the semantic similarity of their `plot_emb` to the first query intent. For each retrieved candidate, it performs a dependent K_j-NN lateral join ($K_j = 5$) against T_2. Crucially, the join condition is a dynamic multi-scoring aggregation that combines the distance between T_2's `plot_emb` and the second query intent, alongside the distance between the `actor_director_emb` vectors of T_1 and T_2. The interpolation weights sum to 1.0. This workload benchmarks the engine's capability to execute complex, late-binding multi-scoring functions nested within a dependent lateral join.

```
-- Q: Find "seasoned detective" plots in T1, then join T2
     movies matching
-- a "rookie cop" plot AND having similar cast/crew
SELECT t1.title AS t1_title, t2.title AS t2_title
FROM (
  SELECT tconst, title, actor_director_emb
  FROM imdb_T1
  ORDER BY plot_emb <=> '[0.012, -0.005, ...]'
  -- plot intent 1
  LIMIT 20
) t1
CROSS JOIN LATERAL (
  SELECT tconst, title
  FROM imdb_T2 t2
  ORDER BY
    0.10 * (t2.plot_emb <=> '[0.033, -0.011, ...]')
    -- plot intent 2
  + 0.90 * (t2.actor_director_emb <=> t1.
     actor_director_emb)
    -- cast similarity join
  LIMIT 5
) t2;
```

Listing 8: Example W7 Query

W8: Pair up similar-cast films across two plot themes, within genre/era on each side (Filtered multi-scoring join). Building upon W7, workload W8 represents the most comprehensive query pattern, introducing exact structural predicates ($\mathcal{P}$) on both sides of the multi-scoring join to evaluate filtered inter-entity synthesis. We synthesize 100 pairs of distinct movie concepts using Claude Opus 4.6 (temperature = 0.8) with the strict prompt: *"Generate exactly [N] pairs of movie concepts. Each line should contain two distinct concepts separated by a pipe character '|'. For example: 'A gritty sci-fi thriller about time travel | A lighthearted comedy about a robot learning to love'. Output only the pipe-separated pairs."* Each text field is independently vectorized into a 1536-dimensional embedding using the Gemini-embedding-001 model. To construct the structured predicates, each query assigns a random temporal bounding interval to both the driving table and the lateral join target: T_1 queries are constrained to eras like [1950, 1969], while the dependent T_2 queries are bounded by modern ranges like [2017, 2020]. During execution, the system first retrieves the top $K = 20$ candidate movies from T_1 satisfying both the T_1 year filter and the semantic similarity of their `plot_emb` to the first query intent. For each retrieved candidate, it performs a dependent K_j-NN lateral join ($K_j = 5$) against T_2, enforcing the T_2 year filter. The join condition is a dynamic multi-scoring aggregation combining the distance between T_2's `plot_emb` and the second query intent, alongside the distance between the `actor_director_emb` vectors of T_1 and T_2. This workload serves as the ultimate stress test, benchmarking the engine's capability to execute complex, late-binding multi-scoring functions nested within a dependent lateral join while simultaneously enforcing relational filters at multiple stages.

```
-- Q: Find "cynical detective" plots in T1 (1950-1969),
     then join T2 movies
-- (2017-2020) matching a "naive rookie cop" plot AND
     having similar cast
SELECT t1.title AS t1_title, t2.title AS t2_title
FROM (
  SELECT tconst, title, actor_director_emb
  FROM imdb_T1
  WHERE startYear >= 1950 AND startYear <= 1969
  ORDER BY plot_emb <=> '[0.024, -0.015, ...]'
  -- plot intent 1
  LIMIT 20
) t1
CROSS JOIN LATERAL (
  SELECT tconst, title
```

```
  FROM imdb_T2 t2
  WHERE t2.startYear >= 2017 AND t2.startYear <= 2020
  ORDER BY
    0.15 * (t2.plot_emb <=> '[0.018, 0.033, ...]')
    -- plot intent 2
  + 0.85 * (t2.actor_director_emb <=> t1.
     actor_director_emb)
    -- cast similarity join
  LIMIT 5
) t2;
```

Listing 9: Example W8 Query

8.2 MOLECULE Benchmark

8.2.1 Dataset Description. The MOLECULE dataset is designed to simulate a complex biomedical knowledge discovery scenario, enabling cross-entity retrieval between isolated chemical facts and academic literature. To facilitate scenarios such as finding empirical literature that supports specific biological properties for a class of molecules, we partition the knowledge graph into two distinct relational tables: `molecule_fact` and `molecule_paper`. Each record represents either a structured biological assertion or a scientific publication, encapsulating both categorical metadata (e.g., organisms, journal tiers) and high-dimensional semantic embeddings of their textual content.

8.2.2 Data Source and Construction. To construct this benchmark, we extracted and integrated a sub-graph from real-world biomedical knowledge repositories (e.g., combining chemical databases like PubChem with literature hubs like PubMed):

- **Data Sourcing:** We sourced raw biological assertions and academic paper metadata from public biomedical knowledge graphs. To ensure data quality and relevance, the extraction was filtered to include well-defined property relationships and peer-reviewed publications with complete abstracts.
- **Profile Construction:** The dataset is divided into two primary entities. For the `molecule_fact` table, we extracted structured attributes including the target `organism` (e.g., Human, Mouse, *E. coli*) and `property_type` (e.g., Toxicity, Target, Pathway), alongside a natural language description of the fact. For the `molecule_paper` table, we retained relational metadata such as publication `year` and `journal_tier` (e.g., Q1, Q2), along with the paper's title and abstract. Crucially, for both tables, we synthesized a textual profile describing the primary molecule involved in the fact or studied in the paper.
- **Vectorization / Partition:** We generated 1536-dimensional embeddings for all textual fields using the Gemini-embedding-001 model. The resulting database consists of the `molecule_fact` table (178,396 cleaned records), which contains embeddings for the fact description (`fact_emb`) and its subject molecule (`molecule_emb`); and the `molecule_paper` table (245,182 records), which contains embeddings for the abstract (`abstract_emb`) and the primary molecule studied (`paper_molecule_emb`). This dual-table architecture provides the physical target for semantic joins, allowing the system to link facts to papers by computing the geometric distance between `molecule_emb` and `paper_molecule_emb`.

9 Integration with Semantic Operator Systems

As established in Sec. 2.1, DASE is designed to assist *semantic operator systems* [30, 36, 40] by acting as a cost-effective prefiltering stage that prunes input data before any LLM invocation. These systems typically implement a suite of LLM-backed *semantic operators*, including SEM_FILTER, SEM_CLASSIFY, SEM_RANK, SEM_JOIN, and SEM_MAP. DASE functions as a plug-in prefilter for these operators, leveraging multi-facet vector embeddings to identify the most promising candidate tuples for downstream LLM evaluation. Detailed SemBench performance results are provided in Table 3.

9.1 The Prefilter Pipeline

In this architecture, DASE serves as a pluggable prefilter component. The overall workflow typically operates in two stages: DASE first recommends a high-recall candidate set, which a full-featured semantic operator engine subsequently evaluates and refines using targeted LLM calls. Alternatively, in scenarios where the downstream semantic operator is omitted, the workflow bypasses LLM evaluation entirely and simply accepts all of DASE's multi-dimensional recommendations directly as the final output. This "DASE alone" configuration (denoted as *DASE* in Table 3) remains highly competitive whenever the embedding distance effectively separates relevant tuples from the background distribution.

9.2 Per-Operator Strategies

The integration logic is specifically tailored to the semantics of each operator. Crucially, DASE treats every natural-language query as a *multi-dimensional evaluation task*: it decomposes the user's semantic intent into a statistical proxy comprising weighted aggregations over multiple, potentially heterogeneous embedding streams (e.g., visual, textual, and structural). At the execution layer, these operators are natively compiled into the multi-scoring architecture introduced in Sec. 4, where independent Filtered-Score Streamers (FSS) are synchronized and pruned by the Multi-Scoring Aggregator (MSA).

9.2.1 SEM_FILTER. A SEM_FILTER evaluates a natural-language Boolean predicate ℓ on an input relation T. For a query such as "finding a blue shirt with positive reviews," the semantics are not restricted to a single modality. DASE captures this by modeling the filter as a joint multi-scoring task across all relevant feature spaces $\mathcal{F}$ (e.g., image embeddings for visual traits and text embeddings for sentiment).

For each space $f \in \mathcal{F}$, DASE derives a specific set of positive anchors P_f and antithetical anchors N_f. The MSA then synchronizes these independent FSS streams to compute a cross-dimensional contrastive margin on the fly:

$$s_i = \sum_{f \in \mathcal{F}} w_f \left(\frac{1}{|P_f|} \sum_{p \in P_f} \cos(\mathbf{e}_p, \mathbf{e}_i^{(f)}) - \frac{1}{|N_f|} \sum_{n \in N_f} \cos(\mathbf{e}_n, \mathbf{e}_i^{(f)}) \right)$$

where $\mathbf{e}_i^{(f)}$ is the embedding of tuple i in space f, and w_f balances the contribution of each modality. This mechanism natively aggregates signals—such as visual blueness and textual sentiment—to identify top candidates. Because DASE acts as a high-recall prefilter, avoiding false negatives is paramount. We employ two recommendation rules that utilize intentionally relaxed boundaries:

- *Margin-based uncertainty band.* For total labeling tasks (e.g., *Wildlife* Q1), DASE applies thresholds to the joint margin s_i. To aggressively protect recall, it forwards a conservatively wide uncertain band (typically $\alpha \approx 0.2$–0.5) for LLM evaluation, while accepting the cheap sign of s_i for confidently extreme rows.
- *Top-K′ shortlist.* For `LIMIT` K queries (e.g., *Movie* Q1), the MSA seamlessly streams the K' tuples with the highest aggregate s_i. The downstream engine's `AI.IF` then evaluates this multi-dimensional shortlist, short-circuiting at the K-th valid survivor. This heuristic reduces the query latency for *Movie* Q1 from 26.3 s to 0.5 s.

9.2.2 SEM_CLASSIFY. A SEM_CLASSIFY operator assigns each tuple to one of K enumerated classes $C = \{c_1, \ldots, c_K\}$. Whereas SEM_FILTER contrasts a single positive concept against its antithetical paraphrases, SEM_CLASSIFY generalises this to a K-way contrast: each class becomes its own anchor, and the per-row decision reduces to identifying which anchor wins. For instance, in *Cars* Q10 the engine must classify automotive complaints into 24 distinct problem categories, where a complaint may comprise both a textual description and an image of the defect, requiring multi-modal reasoning.

DASE natively maps this multi-class evaluation onto a parallel multi-scoring retrieval task. It embeds each class anchor prompt c_k once per facet to obtain $\mathbf{e}_{c_k}^{(f)}$, initialising an independent FSS stream per category. The MSA then synchronises these K streams across the required feature spaces $\mathcal{F}$ (e.g., visual embeddings of the damaged car part and text embeddings of the complaint description), aggregating per-facet cosine similarities into a per-class affinity $s_i^{(k)}$, from which a proxy label $\hat{k}_i$ and confidence margin m_i follow directly:

$$s_i^{(k)} = \sum_{f \in \mathcal{F}} w_f \cos\!\left(\mathbf{e}_{c_k}^{(f)}, \mathbf{e}_i^{(f)}\right), \; k = 1, \ldots, K,$$

$$\hat{k}_i = \arg\max_k s_i^{(k)}, \; m_i = s_i^{(\hat{k}_i)} - \max_{k \neq \hat{k}_i} s_i^{(k)}.$$

All three quantities are computed without any LLM invocation. DASE then partitions the result set on m_i: the bottom-α tuples (smallest margin) are forwarded to the downstream engine for rigorous refinement via `AI.CLASSIFY`, while the engine safely accepts the cheap argmax $\hat{k}_i$ on the remaining confident rows. The final output unions the LLM-resolved tail with the DASE-asserted confident bulk, achieving macro F_1 parity with an exhaustive all-LLM baseline at roughly half the inference cost (Table 3).

9.2.3 SEM_RANK. A SEM_RANK operator orders or scores tuples according to a defined rubric. DASE compiles this requirement into two distinct multi-scoring execution paths based on the query objective:

- *Top-K retrieval.* When the query strictly demands the highest-ranking candidates, the execution naturally reduces to the SEM_FILTER + `LIMIT` paradigm of Sec. 9.2.1. DASE leverages the MSA to synchronise contrastive margin streams across all targeted feature spaces $\mathcal{F}$, organically short-circuiting the retrieval once K confidently top-ranked tuples survive downstream `AI.IF` validation.
- *Rubric scoring.* When a strict numeric label is required for *every* row (e.g., *Movie* Q9, which scores from 1 to 5 how much a reviewer liked a movie), the task is structurally a SEM_CLASSIFY over an *ordinal* class set: each rubric grade $g \in \{1, \ldots, G\}$ plays the role of a class anchor c_g, and the per-grade affinity $s_i^{(g)}$, proxy label $\hat{g}_i = \arg\max_g s_i^{(g)}$, and confidence margin m_i all carry over verbatim from the formulation in Sec. 9.2.2. The only operator-specific change lies in the cascade policy. Because global rank-correlation metrics like Spearman integrate over the entire score distribution, a single mis-ranked tuple can disproportionately degrade the global metric, so DASE cannot afford the aggressive accept-rate used for nominal classification. The bottom-α tail (smallest m_i) is therefore widened substantially ($\alpha \approx 0.7$) and refined via `AI.SCORE` rather than `AI.CLASSIFY`; the proxy argmax $\hat{g}_i$ is accepted only on the confident head, and the two streams are concatenated into the final ranking.

9.2.4 SEM_JOIN. A SEM_JOIN evaluates a match predicate over pairs $(t_i, t_j) \in T_1 \times T_2$, such as verifying if two items share the same brand and category (*Ecomm* Q7). Unlike the per-row operators above, the candidate space here is the Cartesian product $|T_1| \cdot |T_2|$, so prefiltering is not merely an optimisation but a feasibility requirement. DASE evaluates the match proxy directly against the SemJI, which strictly decouples continuous multi-facet similarities in `SemJI(embed)` from discrete semantic truths in `SemJI(AI.IF)`.

During execution, DASE first probes `SemJI(AI.IF)` to resolve previously verified pairs at zero LLM cost. For unresolved pairs, the MSA translates the NL match criteria into a multi-scoring problem, synchronising the traversal across multiple `SemJI(embed)` streams (e.g., product titles *and* images) to compute a joint similarity score.

- *Two-sided uncertainty band.* For joins like *Ecomm* Q7, the MSA organically identifies pairs with the highest aggregate scores ($S_{ij} \geq \tau_{\text{hi}}$) as confident matches. Pairs in the intermediate range ($\tau_{\text{lo}} < S_{ij} < \tau_{\text{hi}}$)—where combined visual and textual cues are strong but exact identity is ambiguous—are forwarded to the LLM. The MSA traversal short-circuits once the upper bound of the joint score drops below τ_{lo}, implicitly pruning the massive tail of certain negatives. If the materialised caches exhaust before reaching τ_{lo}, DASE gracefully resumes the stream by dynamically evaluating distances via the base vector indices as discussed in Sec. 3.5.2.
- *Top-K poles.* For queries seeking extreme pairs (e.g., *Movie* Q5), the MSA fetches the K highest- and lowest-scoring pairs by directly aggregating the underlying facet streams.

Crucially, as the downstream LLM resolves uncertain cases via `AI.IF`, the deterministic results are written back exclusively to `SemJI(AI.IF)`, effectively amortising the cost of multi-faceted reasoning for subsequent queries.

9.2.5 SEM_MAP. A SEM_MAP call applies a per-row natural-language transformation that returns a free-form string, e.g., brand extraction (*Ecomm* Q3), colour labelling (*Ecomm* Q12), or genre clustering (*MMQA* Q4). Because the output domain is open-vocabulary, neither the contrastive margin of SEM_FILTER nor the K-way argmax of SEM_CLASSIFY applies—there is no fixed anchor set to contrast

against. Instead, DASE exploits a complementary form of semantic redundancy: tuples whose embeddings are near-duplicates across the query-relevant feature spaces $\mathcal{F}$ are statistically guaranteed to map to the same target string, so a single LLM call on one representative safely labels its entire neighbourhood.

To compose this joint similarity, the MSA initialises a parallel FSS per facet $f \in \mathcal{F}$ and aggregates the per-facet cosine streams into a unified affinity $\sigma_{ij} = \sum_{f \in \mathcal{F}} w_f \cos(\mathbf{e}_i^{(f)}, \mathbf{e}_j^{(f)})$ on the fly. DASE then clusters the input tuples by agglomerative clustering on this aggregate affinity (complete linkage, distance threshold τ_d=0.10, i.e., $\sigma_{ij} \geq 0.90$) and recommends only the medoid nearest each cluster centroid. The downstream engine runs `AI.GENERATE` on this condensed K-tuple shortlist (e.g., ∼90 medoids out of 500 rows on *Ecomm* Q3), and the returned label is broadcast to the entire cluster via a SQL join on the synthetic cluster ID. Because clusters form in the joint multi-facet space rather than over any single embedding axis, the medoid is robust to single-modality drift—visually similar but textually distinct items remain in separate clusters. Quality degrades only when τ_d is lax enough to fuse genuinely distinct entities, which on *Ecomm* costs less than 0.01 ARI while reducing LLM invocations by an order of magnitude.